Web Development Notes

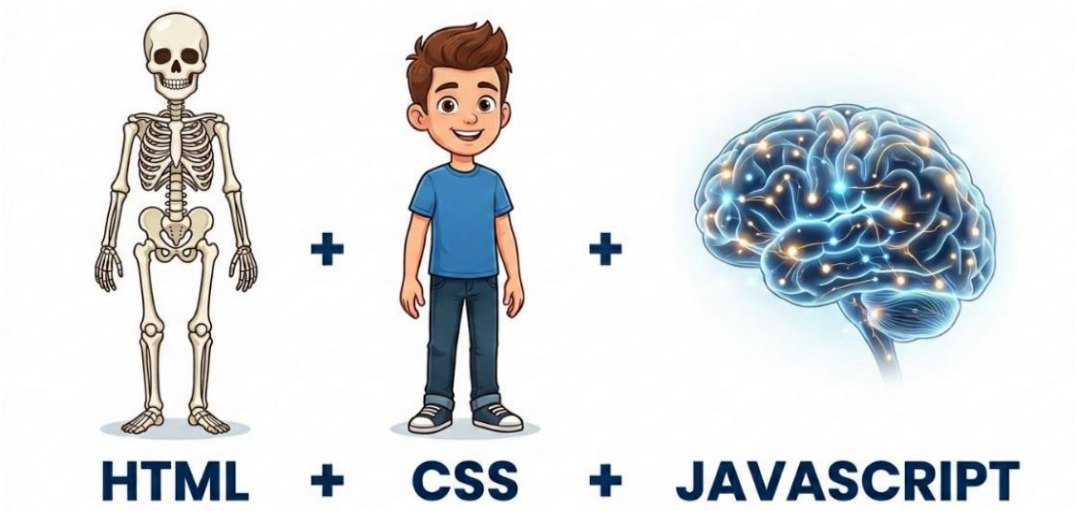
Whenever you open a website like Google or YouTube, you see buttons, images, text, menus, videos and many other visual elements on your screen. It looks clean and easy to use because it is designed using **Graphical User Interfaces (GUI)**. But behind this graphical view, there is a large amount of code working silently in the background. This code tells the browser what to show, how to show it, and how users can interact with it.

Every website you see on the internet is built using different web technologies. The most basic and important technologies are **HTML, CSS, and JavaScript**. These three technologies work together to build the structure, design, and behaviour of a website. The browser reads these codes and converts them into the visual web page we see on our screen.

To build any website, three main technologies work together:

- **HTML** – **(Hyper Text Markup Language)** Which builds the structure of the webpage.
- **CSS** – **(Cascading Style Sheets)** Which designs and styles the webpage.
- **JS** – **(JavaScript)** Which adds interaction, functionality and behaviour to the webpage.

You can imagine a website like a **human body**. HTML acts as the **skeleton**, CSS works like the **skin, muscles, and clothes**, and JavaScript behaves like the **brain** that controls actions and responses.



HTML provides the basic structure of the webpage. It defines: Headings, Paragraphs Images, Links, Sections. Without HTML, a webpage would not have any organized content.

CSS is used to style the webpage. It controls things like: Colores, Fonts, Layout, Spacing, Background images.

JavaScript makes websites interactive. It allows websites to: Respond to clicks, validate forms, show popups, Create animations, Update content dynamically.

Combining all three creates a meaningful website. All these technologies are used to create the frontend part or the part we saw as a viewer of a website. At first, we will learn about HTML.

What is HTML?

HTML stands for **Hyper Text Markup Language**. HTML is the standard language used to create webpages. It tells the browser what content should appear on the webpage such as headings, paragraphs, images, links, and other elements.

HTML does not focus on styling or behaviour. Its main job is to organize and structure the content of a webpage.

The name HTML has three important parts.

- **Hyper Text**

Hyper Text means text that contains links to other pages or documents. Or the text that travels in hyper speed.
Example: When you click a link on a website and it takes you to another page, that is Hyper Text in action.

- **Markup**

Markup means adding special tags to text to describe how it should appear.

- **Language**

HTML is called a language because it has its own rules and syntax that developers follow to create webpages.

What are HTML Tags?

HTML works using **tags**. Tags are special keywords placed inside **angle brackets**: < >
They tell the browser **how to treat the content inside them**.
Example:
This is a paragraph
Here:

- → opening tag
- → closing tag

The text between them becomes a **paragraph**.

If we take an example of **Microsoft Excel**, we have written formulas like:
=SUM(B3:H3)

This formula tells Excel to add numbers from cell B3 to H3 which is inside brackets ( ). Similarly, HTML tags tell the browser how to display certain content which is inside < >.
Example:
<h1>Welcome</h1>

This tells the browser that **"Welcome"** is a **main heading**. So just like Excel uses brackets ( ) in formulas, HTML uses **tags**. < >

How Does a Browser Understand HTML?

A **web browser** (like Chrome, Edge, Brave, or Safari) reads HTML code and converts it into the visual page we see.

Basic Process:

1. A user enters a website address (URL) in the browser.
2. The browser loads the HTML file of that website.
3. The browser reads the HTML line by line.
4. It understands the tags and renders the content on the screen.

Example HTML Code:

```
<h1>Welcome to My Website</h1>
<p>This is my first webpage.</p>
```

What the Browser Shows:

The browser does not display the tags, but it understands them and formats the content accordingly.

***Note:**

HTML is NOT a Programming Language

Many beginners think HTML is a programming language, but it is not. HTML is a Markup Language.

- **Programming languages** (like Java, Python, JavaScript): Perform calculations, make decisions, use logic and conditions.
- **Markup languages**: Describe the structure of content, use tags to label elements.

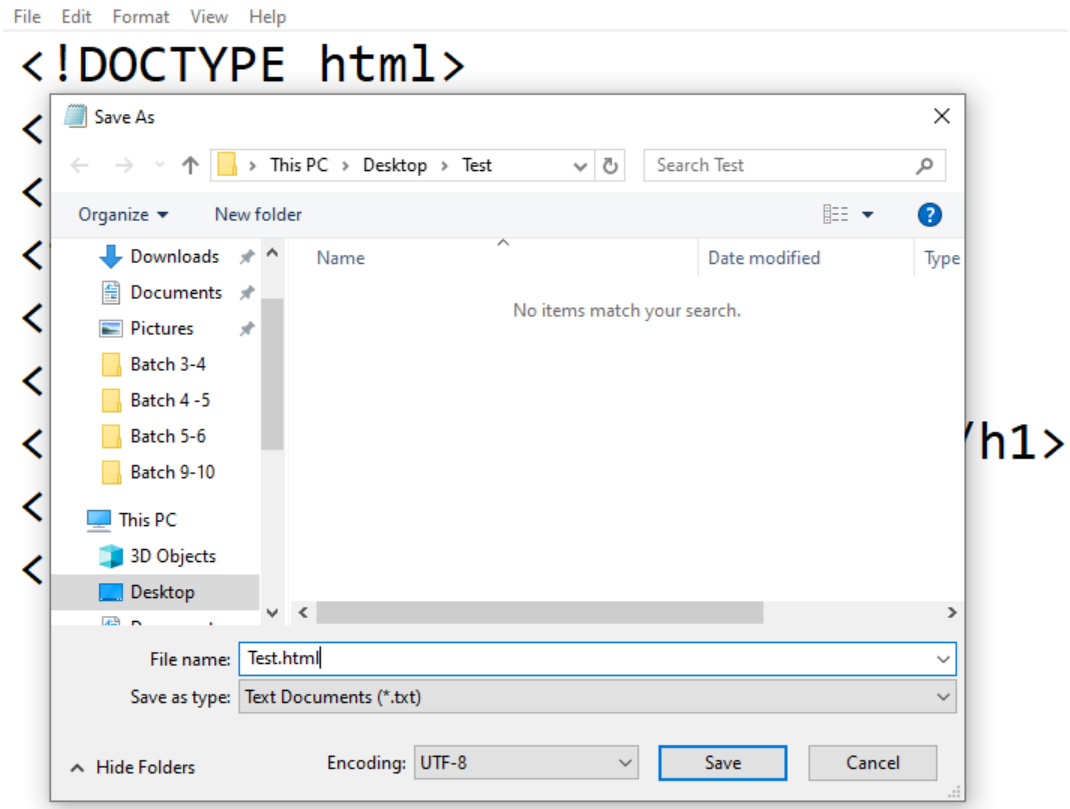
How to Write and Save an HTML file?

As we already knew, a MS Word file uses **.docx** as its extension, a MS Excel file uses **.xlsx** as its extension, a notepad file uses **.txt** s its extension. The same way a HTML file uses **.html** as its extension. To write a HTML file we can use a Text Editor or an IDE.

- **Text Editor:** A text editor is a lightweight tool designed for speed and simplicity. It primarily used for writing and modifying plain text or Code. **Example: Notepad in Windows, Notepad++, Vim etc.**
- **IDE (Integrated Development Environment):** An IDE is a comprehensive software application that combines an editor with advanced tools like compilers and debuggers to manage the entire development lifecycle. **Example: Visual Studio Code, Eclipse, PyCharm, IntelliJ Idea etc.**

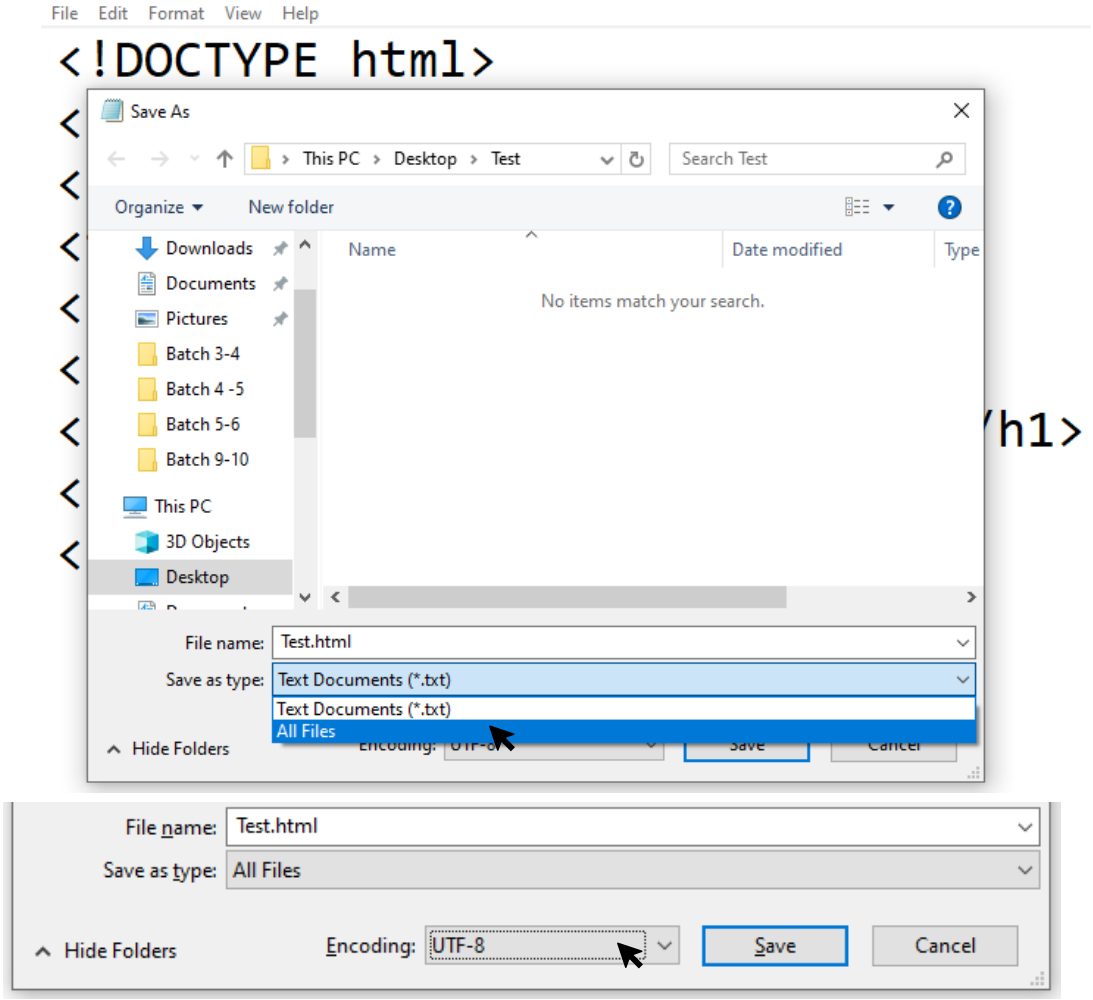
For our lessons, we will be using Notepad or Notepad++ because it's very lightweight, good for beginners and easy to work with.

In this image, there is a HTML code that is written by using Notepad. After writing the code, we need to save it in a different way not like a usual text file.



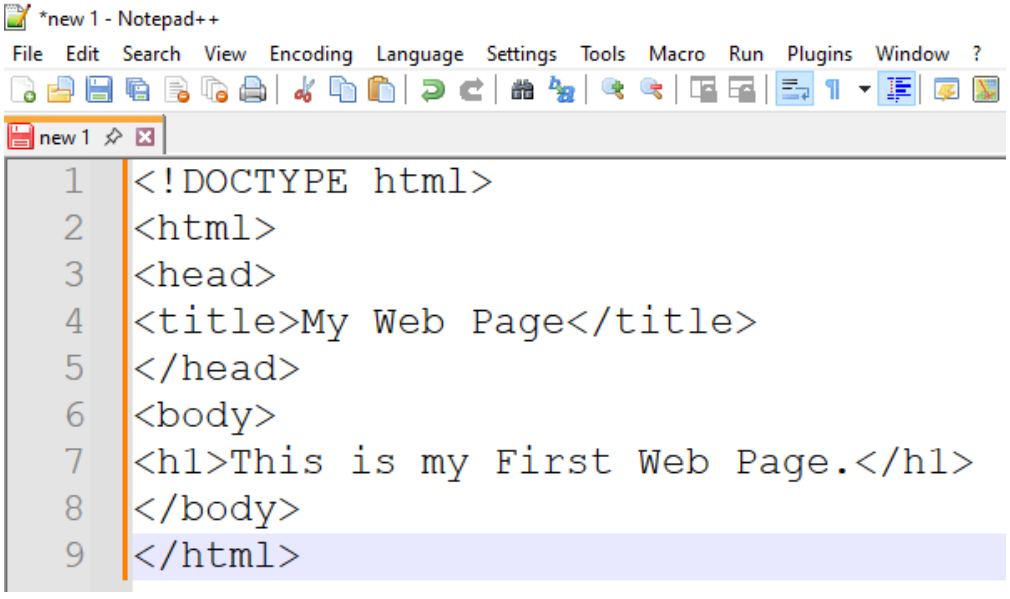
We have to select the folder we want to save the file in and then we have to rename it into our desired name and add .html extension to it. (All HTML files are required to save with adding either .htm or.html extension but the preferred extension is .html).

After that we have to change the filetype in the '**Save as type**' section from **Text Documents(*.txt)** to **All Files**. This is a very important step to follow otherwise our file will be saved as a text file instead of html file.

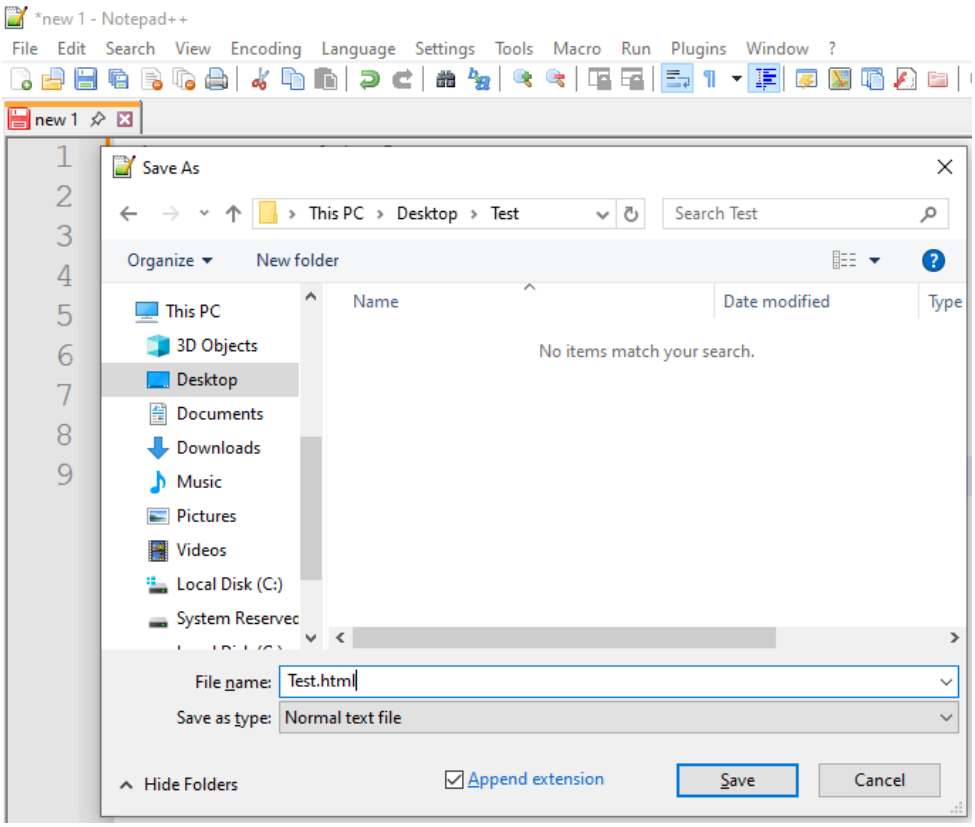


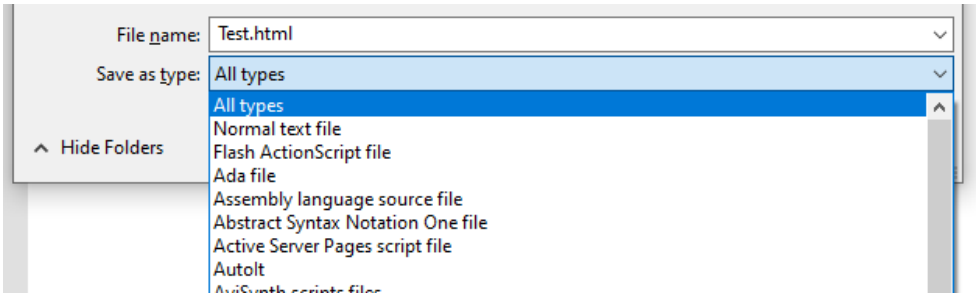
As we are saving the file we need to ensure that the encoding is set to '**UTF-8**'. If there is showing any other encodings like ANSI then we need to change it to UTF-8. UTF-8 is a character set that is responsible to save the HTML file's Encoding that appear on the browser.

In a same way we can code in Notepad++ also.



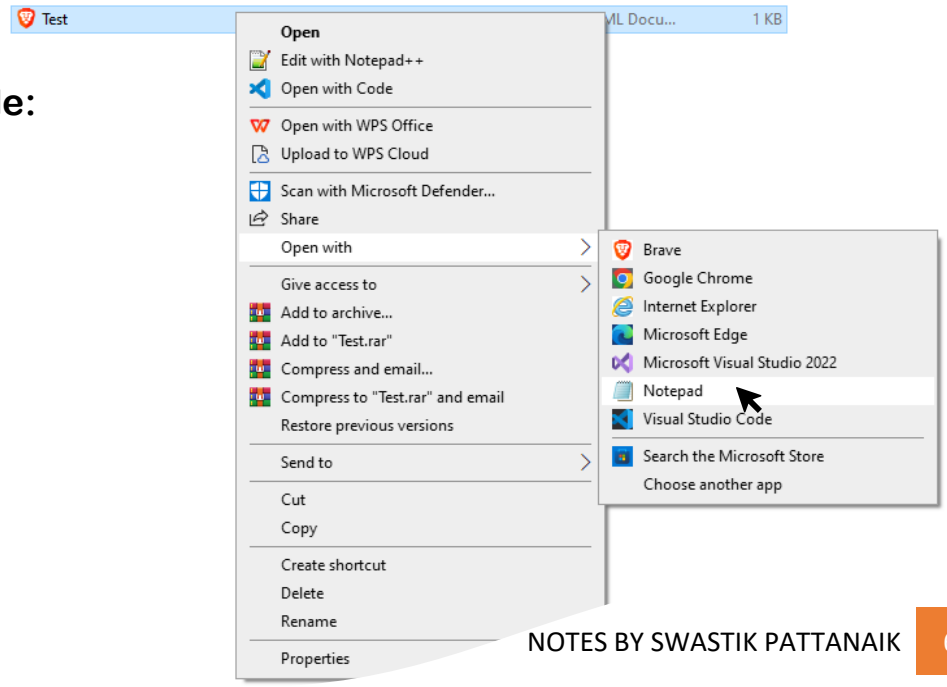
The saving process is also the same as like Notepad. You need to first select the folder you want to save in your file, then you need to give it a name and then you have to use the extension **.html**. After it in the '**Save as type**' section you need to select **All types** from the list. After it you need to click on save. You don't need to select the encoding this time because Notepad++ already taken the charset UTF-8 encoding by default inside the application. To find it you can go to **Encoding tab > UTF-8** in Notepad++.

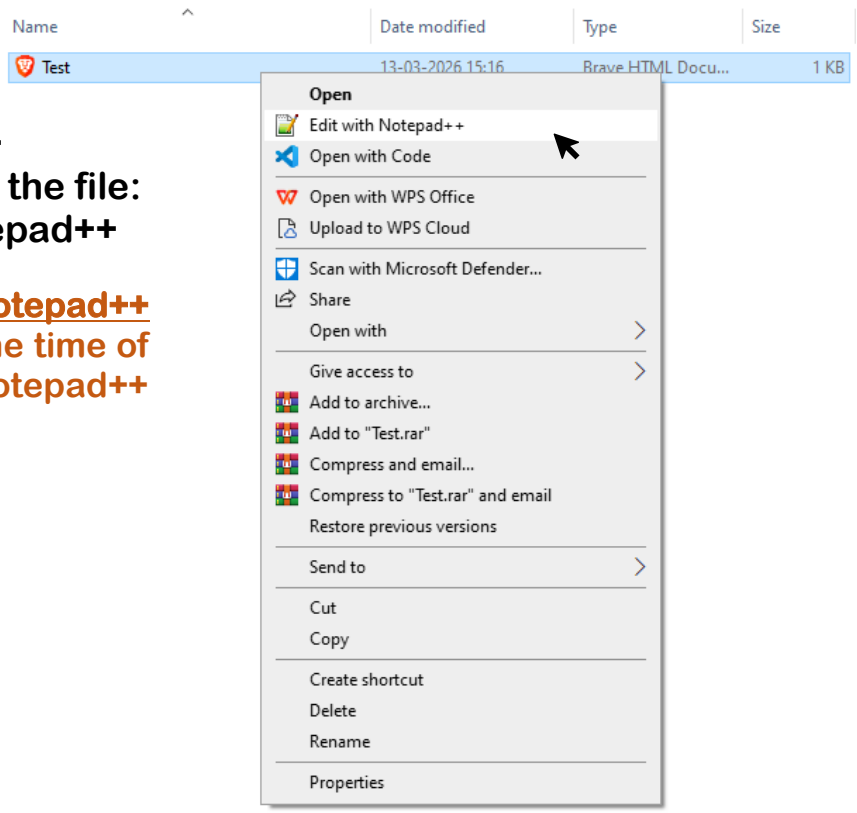




After saving your file, you will see it appearing as a browser file or a web file that can be viewed from browser. If you want to edit the document then can edit it by following methods;

1. For Notepad
   - ➢ Right click on the file:
   - ➢ open with:
   - ➢ notepad

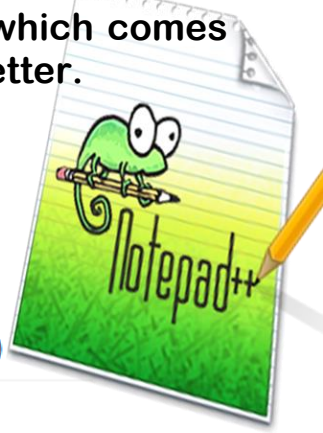




2. For Notepad++
   - ➢ Right click on the file:
   - ➢ Edit with Notepad++

The **Edit with Notepad++** option added at the time of installing the Notepad++ Application.

The difference between Notepad and Notepad++ is just that Notepad is a simple text editor built into Windows. It used for basic tasks like writing notes or editing plain text files. Whereas Notepad++ is a free, advanced text and code editor. It is designed for programmers and power users which comes with a lot of features and facilities. Here's a table that explains better.



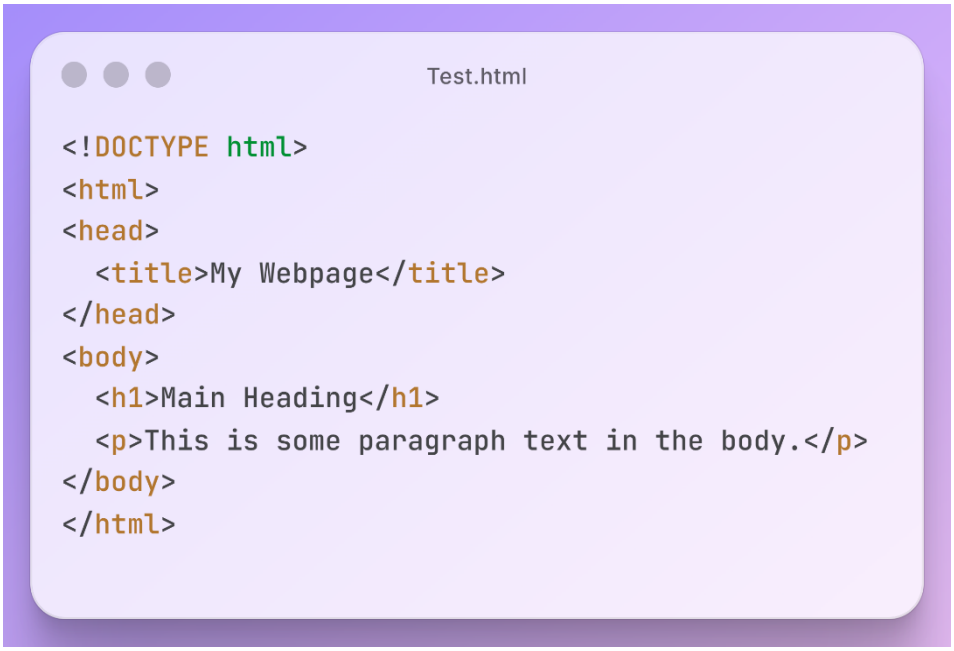
Feature	Notepad	Notepad++
Ease of use	Very simple	Slightly more advanced
Syntax highlighting	❌ No	✅ Yes (for many languages like Python, C++, HTML)
Tabs (multiple files)	❌ No	✅ Yes
Plugins support	❌ No	✅ Yes
Auto-completion	❌ No	✅ Yes
File size handling	Limited	Handles large files better
Customization	Very limited	Highly customizable

Not only HTML, CSS and Java Script can also be written in Notepad++. Apart from these; We can also write many other programming languages like Java, C/C++, Python also by using Notepad++.

Understanding the Basic Structure of a HTML file:

We discussed earlier that an html file consists of different types of tags. But the tags should be written in a well maintained and structured way. The browser understands the structure and builds the website from our code, so maintaining it is very necessary.

A basic html file consists these tags, such as: <!DOCTYPE html>, <html>, <head>, <title>, <body> with some <p> and <h1> tags which creates the basic structure. These tags are written like;



Every Html program should be written according to this structure. Let us Understand each tag one by one.

<!DOCTYPE html>

It is a declaration written at the very top of an HTML document. It tells the web browser that the document is written in **HTML5**, which is the latest version of HTML.

Why do we write <!DOCTYPE html>?

- **Tells the browser how to read the page:**
  It informs the browser that this is a standard HTML document, so the browser should display it using modern rules.
- **Ensures proper rendering (no errors):**
  Without it, browsers may switch to something called *"quirks mode"*, where they follow old and inconsistent rules. This can break your layout or design.
- **Maintains consistency across browsers:**
  Different browsers (Chrome, Edge, Firefox, etc.) will display your webpage correctly and similarly when this declaration is present.

- **It is simple and universal:**
  In HTML5, the doctype is very short and easy: <!DOCTYPE html>. Older versions had long and complex doctypes.

Why we moved from HTML4 to HTML5?

What was HTML4?

HTML4 (released in 1997) was widely used to create web pages. It focused mainly on structuring text and basic layouts. At that time, the internet was simpler, so HTML4 was enough.

❌ Limitations of HTML4

As technology improved, HTML4 started showing problems:

1. **No built-in multimedia support:**
   - To add videos or audio, developers had to use plugins like Flash.
   - These plugins were slow, unsafe, and not supported on many devices.
2. **Poor support for modern applications, language and texts:**
   - Could not handle dynamic and interactive websites easily.
   - Required heavy use of JavaScript and external tools.
   - Supports **ANSI** Character Set.
3. **Lack of semantic structure:**
   - Developers used too many tags.
   - Hard for browsers and search engines to understand content.
4. **Not mobile-friendly:**
   - HTML4 was designed mainly for desktop websites.
   - It didn't support responsive design properly.
5. **Complex and lengthy code:**
   - More code was needed to achieve simple tasks.

✔ Why HTML5 was introduced

HTML5 (released around 2013) was developed to solve all these problems and meet modern web needs.

Key Features of HTML5

1. **Built-in Multimedia Support:**
   - Tags like <video> and <audio> allow media without plugins.
2. **Semantic Elements:**
   - New tags like <header>, <footer>, <article>, <section>
   - Makes code more meaningful and easier to understand.
3. **Mobile-Friendly Design:**
   - Works well on smartphones and tablets.
   - Supports responsive web design.
4. **Better Performance:**
   - Faster loading and smoother user experience.

5. Support for Modern Web Apps, languages and texts:

- Features like local storage, canvas, and APIs.
- Helps in building advanced applications.
- Supports UTF-8 Character Set.

Character Set: ANSI vs UTF-8

What is a Character Set?

A character set defines how characters (letters, numbers, symbols, emojis) are **stored in binary form** so computers can understand and display them.

ANSI (American National Standards Institute)

About ANSI:

- An older encoding system.
- Uses **1 byte (8 bits)** to store characters.
- Can represent only **256 characters.**

❌ Problems with ANSI:

1. Limited characters → Mostly English only.
2. Cannot properly display languages like Hindi, Odia, Telugu, Tamil, etc.
3. Different systems may show different symbols.
4. Not suitable for global use.
5. Emojis support is very less or mostly none.

Example: Odia text may appear as random symbols.

UTF-8 (Unicode Transformation Format - 8 bit)

About UTF-8:

- A modern encoding system based on Unicode.
- Uses **1 to 4 bytes** to represent characters.
- Can represent **millions of characters.**

✔ Advantages of UTF-8:

1. Supports **almost all languages in the world**.
2. Same display across all devices and browsers.
3. Backward compatible with ASCII.
4. Efficient and widely used.
5. All types of emojis are supported.

Due to HTML4 used ANSI and HTML5 used UTF-8 character set, it replaced in in modern web development because it allows websites to display all languages correctly and consistently worldwide in every type of devices.

<html>...</html> HTML Tag

The <html> tag is the **root element** of an HTML document. It tells the browser that everything inside it is part of an HTML webpage. It is the **main container** that holds all other HTML elements.

Example:

```
<!DOCTYPE html>
<html>
    <head>
        <title>My Page</title>
    </head>
    <body>
        Hello World
     </body>
</html>
```

Why <html> Tag is Important?

1. **Root of the Document:** It is the **parent** of all HTML elements, all tags like <head> and <body> are inside it and its **child** tags.
2. **Helps Browser Understand the Page:** Tells the browser: *"This is an HTML document"* and ensures proper rendering of the webpage.
3. **Required for proper structure:** Without <html>, the page structure becomes incomplete. Some browsers may still run it, but it is not correct practice.

<html> tag Attributes:

lang attribute:

It tells the browser the language of the webpage.

E.g.: <html lang="en">

Why it is important:

- Helps search engines (SEO)
- Helps screen readers for accessibility
- Improves user experience

dir Attribute (Direction):

It defines text direction in webpage.

E.g.: <html dir="ltr">

It has two values:

- ltr → left to right (English)
- rtl → right to left (Arabic, Urdu)

Key Points to Remember:

- <html> is the first main tag after <!DOCTYPE html>.
- It wraps the entire webpage content.
- It always has a closing tag </html>.
- Only one <html> tag is allowed per document.

***NOTE:** The **attributes** are a special feature in html. Attributes give **extra functionalities** and **features** to a html tag along with the existing ones.

Ex: <p align="center">

As you can see, Attributes are written inside the starting tag just after the tag name. The comes the (=) is equals to symbol and then the value of the attribute is written inside the (" ") double quotas.

Syntax of the attribute: <TagName AttributeName="AttributeValue">

<head>...</head> Head Tag

The <head> tag contains **information about the webpage**, not the actual visible content. This information is called **metadata** (data about data). Content inside <head> is not displayed on the webpage.

Example:

```
<head>
    <title>My Page</title>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="style.css">
    <style></style>
</head>
```

Why <head> Tag is Important?

1. **Provides Information to Browser:** It helps the browser understand how to load and display the page.
2. **It carries important tags:** In contains tags like title which give a name to the page and also, we can set website icon from here too and also the style tag is written in here.
3. **Improves SEO (Search Engine Optimization):** Search engines read data from <head> and helps your website to rank better.
4. **Links External Resources:** Used to connect CSS, JavaScript, fonts, etc.

Key Points to Remember

- <head> comes before <body>.
- It does not display content directly.
- Only one <head> tag is allowed.

<title>...</title> Title Tag

The <title> tag defines the name of the webpage, which appears on the browser tab and in search engine results.

Example: <title>My Webpage</title>

Important Points

- Displayed on the browser tab.
- Used by search engines → helps in **SEO ranking**.
- Shown when a page is bookmarked.
- Should be short, clear, and relevant.
- Only **one <title> tag** is allowed per page.

In short: It gives the webpage its **identity/name.**

<meta> Tags

The <meta> tag provides **metadata (information about the webpage)** to browsers and search engines. It doesn't affect the webpage.

Example:

```
<meta charset="UTF-8">
<meta name="description" content="This is my website">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Important Uses

- charset="UTF-8" → supports all languages.
- description → helps search engines understand page content.
- viewport → makes the page responsive on mobile devices.

Key Points

- It is a **self-closing tag** (no closing tag).
- Not visible on the webpage.
- Improves SEO and accessibility.

In short: It gives **technical and descriptive information** about the page.

<body>...</body> Body Tag

The <body> tag contains all the visible content of a webpage. Everything that you see in the browser (text, images, links, buttons, etc.) is written inside the <body> tag.

Example:

```
<body>
  <h1>Welcome</h1>
  <p>This is my website</p>
</body>
```

Why <body> Tag is Important?

1. **Displays Content to Users:** All visible elements appear inside <body>, without it, nothing will be shown on the webpage.
2. **Main Working Area:** It is the actual area where we design the webpage which is used to add text, images, videos, forms, etc.
3. **Essential Part of HTML Structure:** Comes after <head> and must be inside <html>.

What Can Be Included Inside <body>

- **Text Elements:** <h1> to <h6> (headings) <p> (paragraphs)
- **Media Elements:** <img> (images) <video> and <audio>
- **Links & Navigation:** <a> (anchor tag)
- **Lists:** , <ol>,
- **Forms:** <form>, <input>, <button>
- **Layout Elements (HTML5):** <header>, <footer>, <section>, <article>

Attributes of <body> Tag:

bgcolor attribute:

This attribute sets the background color of the webpage.

E.g.: <body bgcolor="blue">

text attribute:

This attribute sets the text color of the webpage.

E.g.: <body text="black">

background attribute:

This attribute sets the background image of the webpage.

E.g.: <body background="image.jpg">

***Important Note: These attributes are very old and deprecated in HTML5. So, it's recommended to use CSS instead with the help of style attribute.**

Key Points to Remember

- Only one <body> tag is allowed.
- It contains all visible content.
- Must be placed inside <html>.
- Always has a closing tag </body>.
- Never write the content outside <body>.
- Avoid using multiple <body> tags.
- Avoid using outdated attributes instead use CSS.

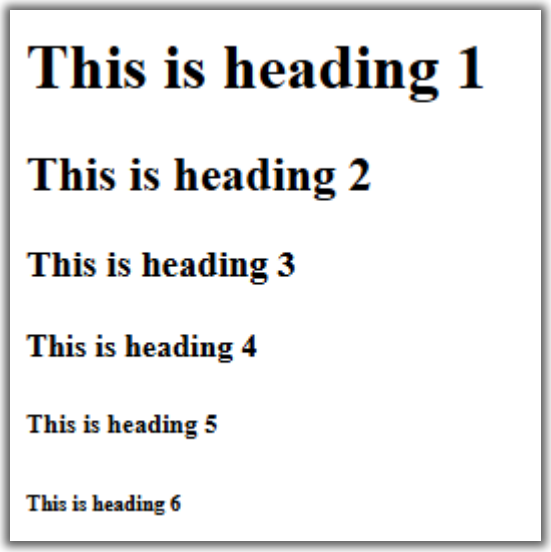
<h1>...</h1> to <h6>...</h6> Heading Tags

Heading tags are used to define **headings or titles** on a webpage. They help organize content and make it easier to read. There are total of six heading tags tags (<h1> to <h6>) that are used to structure and organize content. Each one has their different size of texts and used according to the requirement.

Example:

```
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
```

Output:



Note: The code written above contains only the heading tags but while you are doing practical you need to write the proper structure of the html first including <!DOCTYPE html>, <html>, <head>, <body> etc.

<h1> is the Largest and most important tag and <h6> is the Smallest and least important tag. The size decreases as the number increases. In most of the websites only <h1> to <h4> are getting used.

Key Points to Remember

- There are 6 heading levels.
- Each has an opening and closing tag.
- Used inside <body>.
- Default size is automatically set by browser (which is 32px).

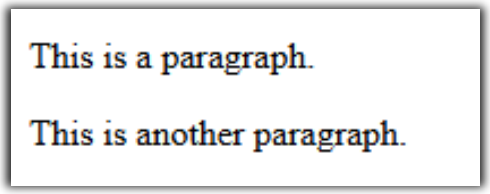
... Paragraph Tag

The tag is used to define a **paragraph of text** in an HTML document. It is one of the most commonly used tags for writing content on a webpage. tag helps to write and organize paragraphs, making the webpage clean, readable, and well-structured. Each tag creates a **new paragraph** and Text inside is displayed in normal font style.

Example:

```
<p>This is a paragraph. </p>
<p>This is another paragraph. </p>
```

Output:



Attributes of Tag

align attribute:

This attribute is used to give alignment to the text in the webpage. Not only tag but also all kinds of other tags also use this attribute.
E.g.: Centered text

It has 3 values in it:

- left → left alignment (Which is the default of every tag)
- center → center alignment
- right → right alignment

Key Points to Remember

- Each has an opening and closing tag .
- Used inside <body>.
- Default size is automatically set by browser (which is 16px).

Types of HTML Tags

In HTML, there are mainly two types of tags that exists:

1. Paired Tags (Container Tags)
2. Unpaired Tags (Empty / Self-Closing Tags)

Paired Tags (Container Tags)

The tags that contain **both an opening tag and a closing tag** are called the Paired tags. It always comes in pairs, closing tag has a forward slash (/) and used when content needs to be displayed or structured.

- **Structure:** <tagname> Content goes here </tagname>.
- **Syntax:** The closing tag is identical to the opening tag but includes a forward slash (/) before the tag name.
- **Purpose:** They define the scope of a specific format or structural element.
- **Common Examples:**
  - <html>...</html>: The root element of an HTML page.
  - ... : Defines a paragraph.
  - <h1> to <h6>: Define heading levels 1 through 6.
  - ... : A generic container for block-level content.
  - <a>...: Defines a hyperlink.

Unpaired Tags (Empty / Self-Closing Tags)

Unpaired tags are HTML tags that **do NOT have a closing tag**. They do not contain any content inside them. These are self-contained tags and typically perform a specific action or embed an element into the page. We can also call it Self Closing tags.

- **Structure:** <tagname> or <tagname />.
- **Syntax:** In HTML5, the trailing slash is optional (
 tag is the same as
 tag, we can write it in both way).
- **Purpose:** Used for standalone elements that do not require internal text or child elements.
- **Common Examples:**
  - <img>: Embeds an image using the src attribute.
  -
: Inserts a single line break.
  - <hr>: Draws a horizontal rule (thematic break).

- <input>: Defines an input field within a form.
- <meta>: Provides metadata about the document.

 Tag (Line Break)

The
 tag is used to **insert a line break** in a webpage. It moves the content to the next line without starting a new paragraph. It is an unpaired (empty) tag that's why it does not require a closing tag.

Example:
Hello
World

Output:

Key Points to Remember

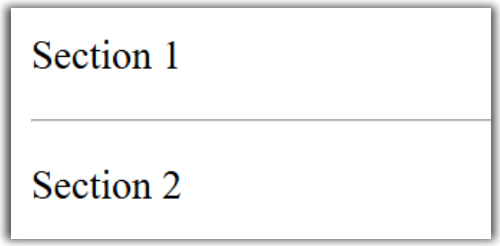
- Used inside text to break lines.
- Does not add extra spacing like .
- It breaks lines without creating a new paragraph.

<hr> Tag (Horizontal Rule)

The <hr> tag is used to create a **horizontal line** across the webpage. It is used to separate content or sections. It is also an unpaired tag and it represents a thematic break in HTML5.

Example:
Section 1
<hr>
Section 2

Output:



Key Points to Remember

- Used between html elements to create a horizontal line.
- By creating separate lines, it creates different sections in webpage.
- It Improving readability by adding visual separation.

Attributes of <hr> Tag

size attribute:

This attribute is used to increase the weight or the thickness of the horizontal rule. The value can be any number. As the number becomes larger the thickness also increases.
E.g.: <hr size="5">

color attribute:

This attribute is used to change the color of the horizontal rule. We can use either the color name or the hex color code.
E.g.: <hr color="red">

***Note:** To apply these attributes in <hr>, we need to use both of them at the same time.
E.g.: <hr size="5" color="red">

Output:

<img> Tag (Image)

The <img> tag is used to **display images** on a webpage. It's also an unpaired tag. Using only the <img> tag won't display the image. We require some specific **attributes** to insert image properly in our webpage.

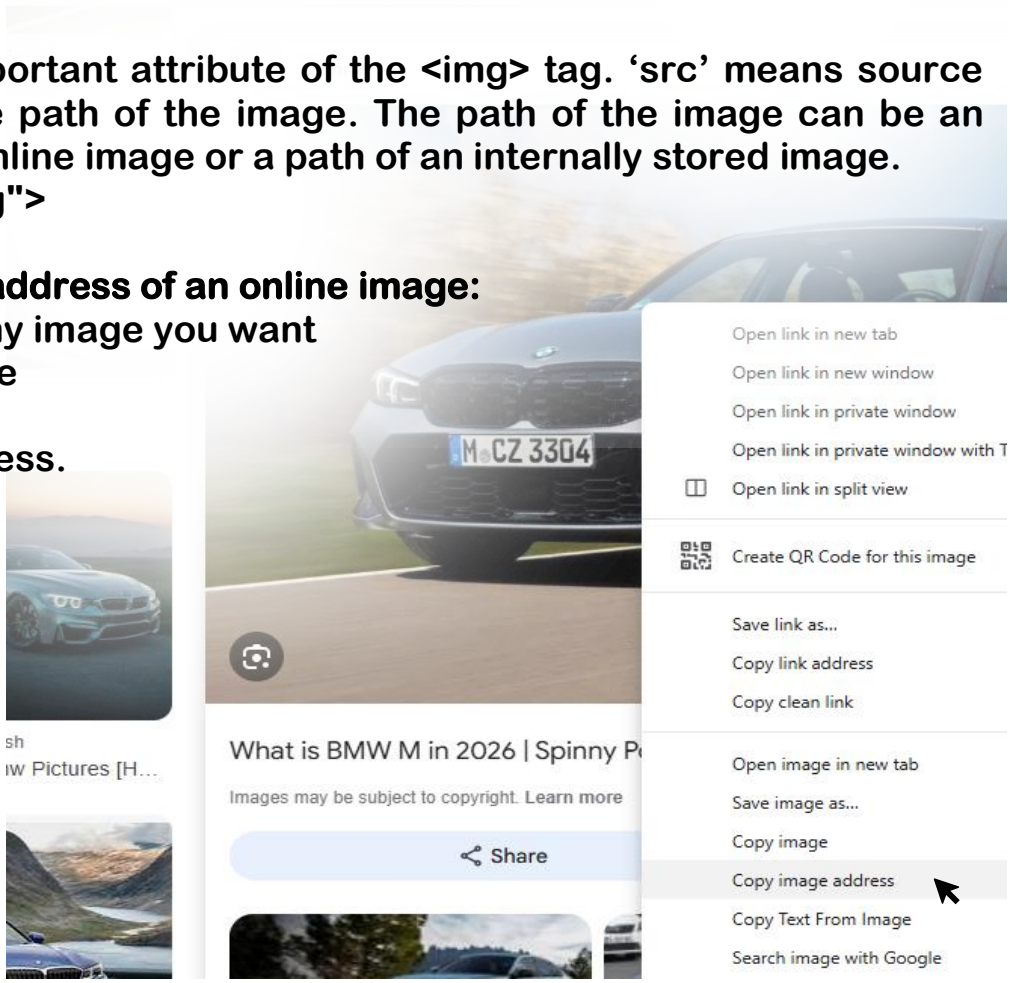
Attributes of <img> Tag

src attribute:

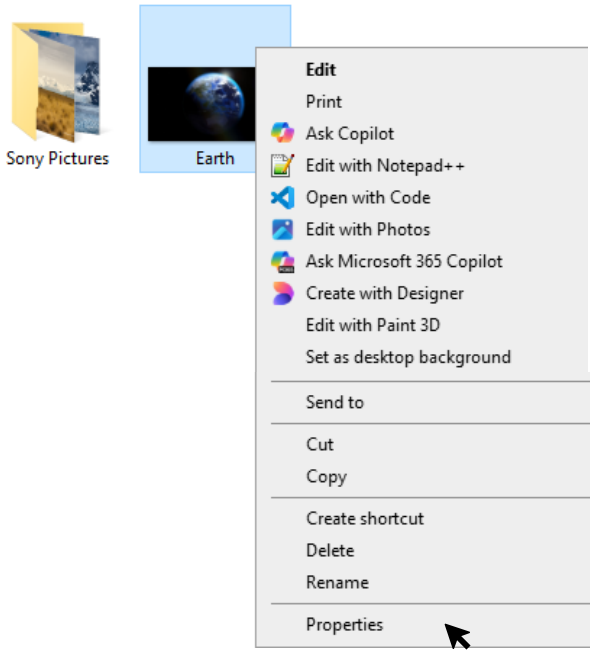
It is the most important attribute of the <img> tag. 'src' means source where we give the the path of the image. The path of the image can be an address or link of an online image or a path of an internally stored image.
E.g.: <img src="car.jpg">

Steps for copying the address of an online image:

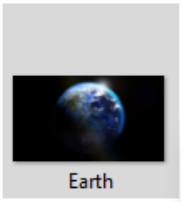
- Google search any image you want
- Click on the image
- Right-Click on it
- Copy image address.



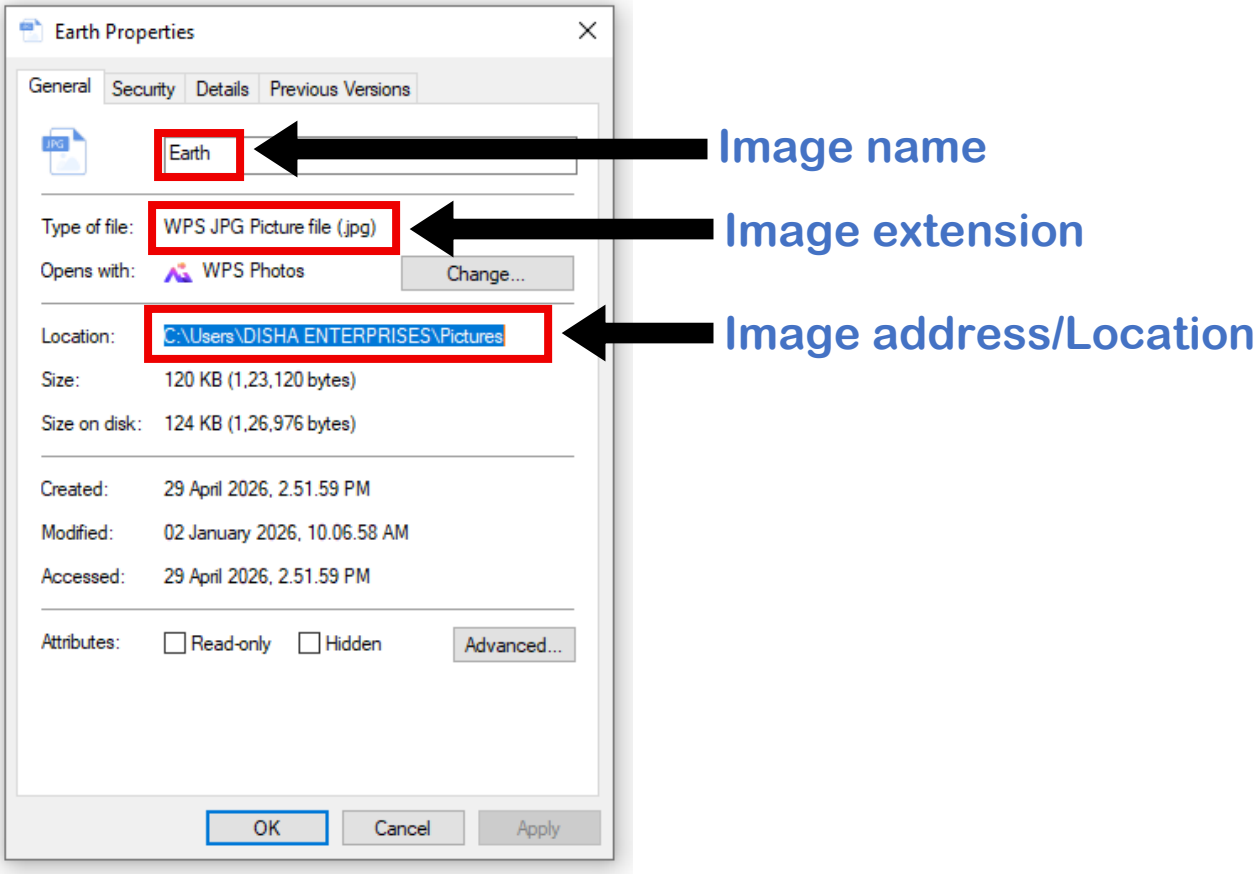
Steps for copying the address of an offline in device image:



- Go to the folder where the image is saved
- Click on the image
- Right Click on it
- Click on Properties
- Copy image address, image name and extension and paste accordingly in src attribute.



At first we need to copy the image address/location then the image name and at last the extension. The pasting order is also same followed by back slashes (\).



height attribute:

Using this attribute, we can give our own custom height to the image. We can give pixels(px) for values in it.

E.g.: <img src="car.jpg" height="200px">

width attribute:

Using this attribute, we can give our own custom width to the image. We can give pixels(px) for values in it.

E.g.: <img src="car.jpg" height="200px" width="300px">

Note: Using the height and width attribute in also necessary in <img> tag because after inserting the image in our webpage, the image will take its original height and width which can be very large or unusually placed in the webpage, so giving our own height and width values are important.

alt attribute:

Using this attribute, we can give a display text if the image is not loaded due to some reasons like if the internet is slow to load the image or the image is deleted. It is important for accessibility and SEO.

E.g.: <img src="car.jpg" height="200px" width="300px" alt="Car Image">

Final Example:

```
<h1>This is image tag</h1>
<img src="https://spn-sta.spinny.com/blog/20230522183738/BMW-M340i-1160x653.webp?compress=true&quality=80&w=1200&dpr=2.6" height="200px" width="300px" alt="Car image">
```

Output:

... Division Tag

The tag is a **block-level container** used to group and organize other HTML elements together. It does not add any special meaning by itself, but it is very useful for layout and styling. The tag is used to group elements, design layouts, and apply styles, making webpages organized, flexible, and easy to manage.

Example:

```
<div>
  <h1>Welcome</h1>
  <p>This is a paragraph inside div</p>
</div>
```

Output:

Here, groups the heading and paragraph into one section. Visually it will be the same as we are giving plain <h1> and <p> tags but actually it combines both of them into one section.

Why tag is important?

1. **Grouping Elements:** Combines multiple elements into one block and makes code more organized and structured.
2. **Styling with CSS:** We can apply same style to multiple elements at once with the help of **style attribute.**
3. **Page Layout Design:** A webpage consists of different section in which the content in managed and it helps to create sections like; Header, Navigation bar, Content area, Footer etc. with the help of **id and class attribute.**
4. **JavaScript Manipulation:** It's very easy to select each section and control using it with JavaScript using **id or class** rather than using each tag individually.

Key Points to Remember

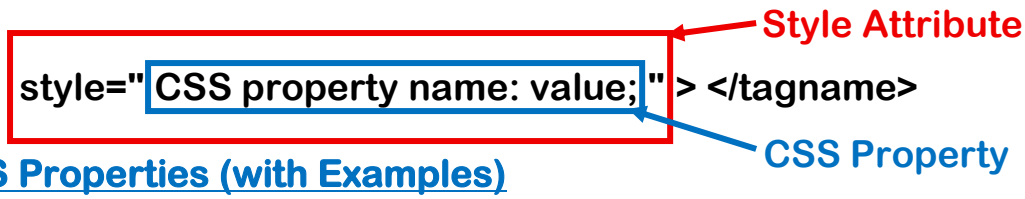
- is a container, not a content tag.
- Must be placed inside <body>.
- Always has a closing tag .
- Improves code readability and easy to understand each section.
- All types of attributes can work in tag.
- Works with CSS and JavaScript.

INTRODUCTION TO BASIC CSS (style attribute)

style Attribute in HTML:

The style attribute is used to apply **CSS (Cascading Style Sheets)** directly to an HTML element. It allows you to change the appearance (design) of elements like color, size, spacing, etc. We can also call it **Inline CSS.** style is an attribute in which we provide different CSS properties as a value.

Syntax:



Basic CSS Properties (with Examples)

color property:

It changes the text color of the tag's content.

E.g.:

```
<p style="color: red;">This is paragraph</p>
```

Output: This is paragraph

background-color property:

It changes the background color of the tag; it only changes the background color of the area that content already occupied.

E.g.:

```
<p style="background-color:
yellow; ">This is paragraph</p>
```

Output: This is paragraph

font-family property:

It changes the font style of the content. We need to give the font name into the value and it changes the font according to it. We can also give multiple fonts also. The default font of webpage is **Times New Roman**.

E.g.:

```
<p style="font-family: Calibri;">This
is paragraph</p>
```

Output: This is paragraph

font-size property:

It changes the font size of the content. We can give the sizes value in pixels(px).

E.g.: `<p style="font-size: 25px;">This is paragraph</p>`

Output: This is paragraph

text-align property:

It changes the alignment of the text just like the align attribute but only for texts. It also has 3 values: left, center, right.

E.g.:

```
<p style="text-align: center;">This is paragraph</p>
```

Output:

border property:

It provides a border to the html element. The border property is also a shorthand in which we can give three property values.

1. We can give the thickness of the border by pixels.
2. We can change the type of the border like: solid, dotted, double, dashed.
3. We can give color of the border.

E.g.:

```
<p style=" border: 2px solid black;">This is paragraph</p>
```

Output:

E.g.:

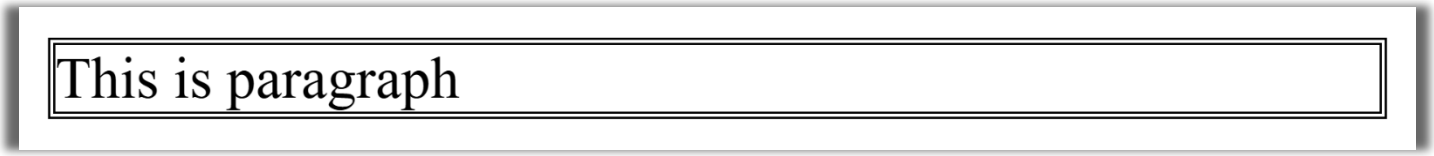
```
<p style=" border: 2px dotted black;">This is paragraph</p>
```

Output:

E.g.:

```
<p style=" border: 2px double black;">This is paragraph</p>
```

Output:



E.g.:

```
<p style=" border: 2px dashed black;">This is paragraph</p>
```

Output:

id and class Attribute in HTML:

In HTML, **id** and **class** are attributes used to identify and style elements on a webpage. They are important for designing, organizing, and managing webpage elements with **CSS** and **JavaScript**. When multiple similar HTML elements are present inside the <body> tag, these attributes help target and distinguish specific elements for styling or functionality. By themselves, **id** and **class** do not create any visible changes; their effect is seen only when they are selected and styled or manipulated using CSS or JavaScript.

1. id attribute:

The **id** attribute gives a **unique name** to a single HTML element. It is unique because it is designed to identify one specific HTML element on a webpage. Just like every student in a school has a unique roll number, every element with an **id** should have its own unique name that can't be repeated in any other element.

E.g.:

```
<div id="Stu1">
    <h1>Student name</h1>
    <p>Student info</p>
</div>
```

Here it is the information o0f an individual student. (Student 1)

Note:

- In CSS, an **id** is selected by using the **hash symbol (#)** before the id name.
- The **id** attribute is important in HTML forms because it connects form elements with their labels.
- It helps organize form data properly for backend processing.
- It allows JavaScript to easily access and interact with specific form elements.

2. class attribute:

The **class** attribute is used to group multiple elements under the same name. Unlike the id attribute, in class attribute we can give multiple tags the same class name. It is one of the most commonly used attributes in HTML because it helps make web pages organized, reusable, and easy to manage.

E.g.:

```
<p class="text">First paragraph</p>
<p class="text">Second paragraph</p>
<p class="text">Third paragraph</p>
```

Here, all three paragraphs belong to the same class **text**.

Note:

- In CSS, a **class** is selected using the **dot symbol (.)** before the class name.
- Multiple class names can be assigned to a single HTML element by separating them with spaces.
- Using classes helps reduce repeated code.
- It also makes webpage design more consistent and easier to manage.

HTML Color Codes

HTML color codes are used to define the colors of text, backgrounds, borders, and other elements on a webpage. They help to make websites look attractive, readable, and visually appealing. Colors in HTML are mainly used through **CSS**, and there are different ways to represent them.

The three common color formats are:

1. **Hex Color Codes**
2. **RGB Colors**
3. **HSL Colors**

1. Hex Color Codes

Hex color codes are the most commonly used method for defining colors in HTML and CSS. A hex color starts with the **hash symbol (#)** followed by **6 hexadecimal characters**.

Note: Hexadecimal character is a number system in computer which is also known as (Base-16) Number system. Which consists of 16 numbers which includes 0 to 9 & A to F.

The color spectrum of Hex color code starts from #000000(black) and goes up to #FFFFFF(white).

Example:

```
<p style="color: #FF0000">This is paragraph</p>
```

This represents the color **Red.**

How it works:

A hex color is divided into three parts: **#RRGGBB**

Where:

- **RR** = Red
- **GG** = Green
- **BB** = Blue

Each value ranges from **00 to FF**.

Examples:

#FF0000 → Red
#00FF00 → Green
#0000FF → Blue
#FFFFFF → White
#000000 → Black

Importance:

Hex codes are widely used by everyone because they are short, precise, and easy to use.

2. RGB Colors

RGB stands for **Red, Green, Blue**. It defines colors by mixing different amounts of red, green, and blue light.

Syntax: rgb(red, green, blue);

Each value ranges from **0 to 255**. Where the 0 value is Black and 255 value is white.

Example:
This is paragraph
This represents the color **Red**.

Examples:

- **rgb(255, 0, 0)** → Red
- **rgb(0, 255, 0)** → Green
- **rgb(0, 0, 255)** → Blue
- **rgb(255, 255, 255)** → White
- **rgb(0, 0, 0)** → Black

Importance:

RGB is useful when you want more control over color mixing.

3. HSL Colors

HSL stands for **Hue, Saturation, Lightness.**

Syntax: hsl(hue, saturation, lightness);

Hue:
Represents the color itself and is measured in degrees from **0 to 360**.
Examples:

- **0°** → Red
- **120°** → Green
- **240°** → Blue

Saturation
Represents color intensity.

- **0%** → Gray
- **100%** → Full color

Lightness
Represents brightness.

- **0%** → Black
- **50%** → Normal color
- **100%** → White

Example:
This is paragraph
This gives **Red**.

Importance:

HSL is easier to understand when adjusting shades, brightness, and tones.

Difference Between Hex, RGB, and HSL

Format	Example	Best Use
Hex	#FF0000	Common web design
RGB	rgb(255,0,0)	Color mixing
HSL	hsl(0,100%,50%)	Adjusting shades

HTML Lists

HTML lists are used to **display related items in an organized way with the help of bullet points** on a webpage. Lists help present information clearly and make content easier to read and understand. HTML lists are very important for organizing webpage content in a structured and readable manner. Different list types are used depending on whether the content needs. There are 3 types of Lists:

1. Ordered List (<ol>...)
2. Unordered List (<ul>...)
3. Description List (<dl>...</dl>)

1. Ordered List (<ol>...)

An ordered list is used when the items need to appear in a **specific sequence or order**. Each item is automatically numbered or lettered by the browser. The ordered lists provide Numerical or Sequential bullet points. It is used when order is important.

For the ordered list we use <ol>... tags. And for mentioning the individual items in the list we use another tag which is <li>... (List Items).

Example:

```
<ol>
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ol>
```

Output:

1. Desktop
2. Keyboard
3. Mouse

The default numerical bullet value of the Ordered list is 1, 2, 3, 4, ... If we want to change the bullet value then we need to use **type** attribute.

type attribute (in Ordered List):

Using this attribute, we can change the type of the numeric bullet in an ordered list. We can set uppercase and lowercase English alphabets and roman letters as ordered list type.

E.g.:

```
<ol type="A">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ol>
```

A. Desktop
B. Keyboard
C. Mouse

```
<ol type="a">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ol>
```

a. Desktop
b. Keyboard
c. Mouse

```
<ol type="I">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ol>
```

I. Desktop
II. Keyboard
III. Mouse

```
<ol type="i">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ol>
```

i. Desktop
ii. Keyboard
iii. Mouse

2. Unordered List (<ul>...)

An unordered list is used when the items need to appear in a **graphical symbol point**. Each item is given a symbol as a bullet point by the browser. The unordered lists provide 4 types of symbols as bullet points. It is used for non-formal lists where sequential order is not necessary.

For the unordered list we use <ul>... tags. And for mentioning the individual items in the list we use another tag which is <li>... (List Items).

Example:

```
<ul>
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ul>
```

Output:

- Desktop
- Keyboard
- Mouse

The default symbolical bullet value of the Unordered list is • **disc** type If we want to change the bullet value then we need to use **type** attribute.

type attribute (in Unordered List):

Using this attribute, we can change the type of the symbol bullets in an unordered list. We can set four types of symbols that are: disc, square, circle and none to unordered list type.

E.g.:

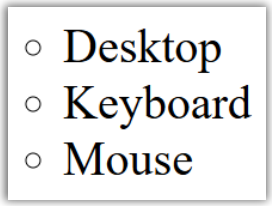
```
<ul type="disc">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ul>
```

- Desktop
- Keyboard
- Mouse

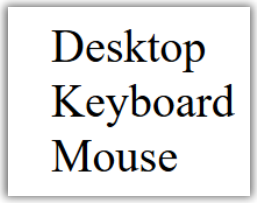
```
<ul type="square">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ul>
```

- Desktop
- Keyboard
- Mouse

```
<ul type="circle">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ul>
```




```
<ul type="none">
    <li>Desktop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ul>
```



In type="none" there will be no bullet points. It mostly used in creation of navigation bars.

3. Description List (<dl>...</dl>)

The <dl> tag stands for **Description List**. It acts as a container that holds terms and their corresponding descriptions. It is mainly used for he purpose of defining a topic which contain a title and its data.

Why We Use It:

- To display a list of terms and their meanings.
- To create glossaries, dictionaries, FAQs, or definitions.
- To show key-value type information in a structured way.

It consists two other tags such as:

1. <dt>...</dt> (Description Title)
2. <dd>...</dd> (Description Data/Details)

1.<dt>...</dt> (Description Title)

The <dt> tag stands for **Description Title**. It is used to define the term, word, or title that will be described. It is used to specify the main term and to identify what is being explained.

1.<dd>...</dd> (Description Data/Details)

The <dd> tag stands for **Description Data/Details**. It contains the description, explanation, or definition of the term written inside the <dt> tag. It's mainly used to provide information about the title and explain the meaning of a word or concept.

E.g.:

```
<dl>
    <dt>HTML</dt>
    <dd>HTML is used to create the structure of a web page.</dd>

    <dt>CSS</dt>
    <dd>CSS is used to style and design web pages.</dd>

    <dt>JavaScript</dt>
    <dd>JavaScript is used to add interactivity to web pages.</dd>
</dl>
```

Output:

HTML
HTML is used to create the structure of a web page.
CSS
CSS is used to style and design web pages.
JavaScript
JavaScript is used to add interactivity to web pages.

HTML Tables

An **HTML Table** is used to display data in a structured formator in a table format consisting of **rows and columns**. Tables are useful when you need to organize and present information such as student records, employee details, marksheets, timetables, product lists, and more.

Unlike paragraphs, where information is written continuously, HTML Table is just like a table in a notebook or Excel sheet, it arranges data into rows and columns, making it easy to read and understand. There are various tags in html table and each tag represents its own work. Also, in HTML Tables the data is inserted row-wise.

Why Do We Use HTML Tables?

- Display tabular data in an organized manner.
- Compare information easily.
- Present records, reports, schedules, and statistics.
- Improve readability of large amounts of data.

Important Table Tags:

1. <table> Tag:

The <table> tag is the main container that creates a table. It holds all rows and columns of the table. Also, it defines the beginning and end of a table. It tells the browser that everything written inside it belongs to a table. Every HTML table always begins with **<table>** and ends with **</table>**.

Why <table> Tag is Important?

1. **Creates the Table:** Without the <table> tag, the browser will not understand that the content should be displayed as a table.
2. **Holds all Table Elements:** It acts as the parent element of all other table tags such as:
   - <tr>
   - <th>
   - <td>
3. **Organizes Data:** It arranges information into rows and columns.

2. <tr>...</tr> Tag (Table Row):

The <tr> tag stands for **Table Row**. Every horizontal row inside a table is created using the <tr> tag. Each row may contain one or more table headings (<th>) or table data (<td>). As in HTML the table data inserted row-wise so the declaration of <tr> tag is necessary before entering the data.

Why <tr> Tag is Important?

- Creates horizontal rows.
- Organizes data row-wise.
- Holds <th> and <td> elements.

3. <th>...</th> Tag (Table Heading):

The <th> tag stands for **Table Heading**. It is used to create heading texts inside the cells of columns or rows inside a table. By default, browsers display the text inside <th> as **Bold** and **Center Aligned Texts.**

Why <th> Tag is Important?

- Identifies headings.
- Makes tables easier to understand.
- Improves accessibility.
- Helps screen readers identify column names.

4. <td>...</td> Tag (Table Data):

The <td> tag stands for **Table Data**. It is used to insert the actual information inside a table. Everything except the headings is generally written using <td>. It enters all the data according to the headings in each cell of each row.

Why <td> Tag is Important?

- It fills the data according to the headings
- Makes the table's data section.
- Helps readers identify data row wise.

E.g.:

```
<table>
    <tr>
        <th>Sl no.</th>
        <th>Name</th>
        <th>Course</th>
        <th>Address</th>
    </tr>
    <tr>
        <td>1</td>
        <td>Rahul</td>
        <td>PGDCA</td>
        <td>Kabisuryanagar</td>
    </tr>
    <tr>
        <td>2</td>
        <td>Smita</td>
        <td>OSCIT</td>
        <td>Aska</td>
    </tr>
    <tr>
        <td>3</td>
        <td>Ashutosh</td>
        <td>O Level</td>
        <td>Kodala</td>
    </tr>
</table>
```

Output:

Sl no.	Name	Course	Address
1	Rahul	PGDCA	Kabisuryanagar
2	Smita	OSCIT	Aska
3	Ashutosh	O Level	Kodala

Attributes of <table> Tag:

HTML tables become more useful using different attributes.

border Attribute:

The **border** attribute is used to display borders around the table and its cells. Without border, the table appears as normal text.

E.g.:

```
<table border="2">
        <tr>
            <th>Sl no.</th>
            <th>Name</th>
            .....
</table>
```

Output:

Sl no.	Name	Course	Address
1	Rahul	PGDCA	Kabisuryanagar
2	Smita	OSCIT	Aska
3	Ashutosh	O Level	Kodala

Note: The table border attribute gives border to each and every cell in the table and to the table itself. Also, if we increase the value of it, only the outer table border will get thicker.

cellpadding Attribute:

The **cellpadding** attribute creates space **inside each table cell**. It increases the distance between the cell border and its content. Without cellpadding, the text appears very close to the borders. As the value increases, the space inside each cell also increases.

E.g.:

```
<table border="2" cellpadding="10">
        <tr>
            <th>Sl no.</th>
            <th>Name</th>
            .....
</table>
```

Output:

Sl no.	Name	Course
1	Rahul	PGDCA
2	Smita	OSCIT
3	Ashutosh	O Level

cellspacing Attribute:

The **cellspacing** attribute creates space **between two adjacent table cells**. Unlike cellpadding, it does **not** increase the space inside the cell. Instead, it separates the cells from one another. Or in simple wors it gives a space to the cell borders. Larger values create larger gaps between cells.

E.g.:

```
<table border="2" cellspacing="10">
        .....
</table>
```

Output:

Sl no.	Name	Course	Address
1	Rahul	PGDCA	Kabisuryanagar
2	Smita	OSCIT	Aska
3	Ashutosh	O Level	Kodala

Difference between cellpadding and cellspacing:

Cellpadding

- Space inside a cell or space around the content.
- Increases distance between border and content.
- Applied inside every cell.

Cellspacing

- Space between cells or space in cell borders.
- Increases distance between two cells.
- Applied between cells.

***Important Note:**

Both **cellpadding** and **cellspacing** are deprecated in HTML5. Today they are replaced using CSS properties such as **padding** and **border-spacing**.

➔ As we earlier discussed about **height, width** and **align** attributes, those are also works on table as it works on other tags.

Colspan and Rowspan Attributes:

colspan Attribute:

Sometimes a single cell needs to occupy multiple column cells. For this purpose, we use the **colspan** attribute. The colspan attribute merges **two or more columns vertically**. Or we can say In Html Table the vertical merging of cells is occur by colspan. We give the colspan attribute in <th> and <td> tags. The value of the attribute is how many cells it going to merge.

E.g.:

```
<table border="2">
        <tr>
            <th colspan="4">Student Table</th>
        </tr>
        <tr>
            <th>Sl no.</th>
            <th>Name</th>
            .....
</table>
```

Output:

Student Table			
Sl no.	Name	Course	Address
1	Rahul	PGDCA	Kabisuryanagar
2	Smita	OSCIT	Aska
3	Ashutosh	O Level	Kodala

Why Colspan is Used?

- Creates a common heading in table.
- Groups related columns with same type of cell data.
- Reduces repetition of data.

rowspan Attribute:

Sometimes a single cell needs to occupy multiple row cells especially in duplicate or similar values. For this purpose, we use the **rowspan** attribute. The rowspan attribute merges **two or more rows horizontally**. Or we can say In Html Table the horizontal merging of cells is occur by rowspan. We also give the rowspan attribute in <th> and <td> tags. The value of the attribute is how many cells it going to merge.

***Imp note:** We need to give rowspan to the uppermost cell/<td> tag among the duplicate values and then need to delete the other values otherwise the table will be unstructured.

E.g.:

```
<table border="2">
    <tr>
<th colspan="4">Student Table</th>
    </tr>
    <tr>
    <th>Sl no.</th>
    <th>Name</th>
    <th>Course</th>
    <th>Address</th>
    </tr>
    <tr>
    <td>1</td>
    <td>Rahul</td>
    <td rowspan="3">PGDCA</td>
    <td>Kabisuryanagar</td>
    </tr>
    <tr>
    <td>2</td>
    <td>Smita</td>
    <td>Aska</td>
    </tr>
    <tr>
    <td>3</td>
    <td>Ashutosh</td>
    <td>Kodala</td>
    </tr> </table>
```

Output:

Student Table			
Sl no.	**Name**	**Course**	**Address**
1	Rahul	PGDCA	Kabisuryanagar
2	Smita		Aska
3	Ashutosh		Kodala

Why Rowspan is Used?

- Avoid repeated information in table.
- Merge vertically related or same cell data.
- Reduces repetition and makes tables cleaner.

HTML5 Semantic Table Tags:

HTML5 introduced semantic tags for tables. These tags make the table more meaningful and easier for browsers, search engines and screen readers to understand. But it doesn't affect the table by visually.

<thead>...</thead> Tag:

The <thead> tag defines the **header section** of a table. It usually contains table headings and column headings written using <th>.

E.g.:
```
<table border="2">
    <thead>
        <tr>
            <th colspan="4">Student Table</th>
        </tr>
        <tr>
            <th>Sl no.</th>
            ……
            <th>Address</th>
        </tr>
    </thead>
```

<tbody>...</tbody> Tag:

The <tbody> tag contains the **main data** of the table. Almost all table rows are generally placed inside <tbody>.

E.g.: ……..
```
    <tbody>
        <tr>
            <td>1</td>
            <td>Rahul</td>
            <td rowspan="3">PGDCA</td>
            <td>Kabisuryanagar</td>
        </tr>
        <tr>……</tr>
        <tr>……</tr>
    </tbody>
    …….
```

<tfoot>...</tfoot> Tag:

The <tfoot> tag defines the **footer section** of a table. It usually contains totals, grand total, average, summary, remarks or some extra info and its written using <td>.

E.g.: ……..
```
            <td>Kodala</td>
            </tr>
    </tbody>
    <tfoot>
        <td colspan="4">The Student's
        Batch is 3 to 4 PM</td>
    </tfoot>
</table>
```

Final output:

Student Table			
Sl no.	Name	Course	Address
1	Rahul	PGDCA	Kabisuryanagar
2	Smita		Aska
3	Ashutosh		Kodala
The Student's Batch is 3 to 4 PM			

HTML Hyperlinks

Whenever we browse the internet, we continuously move from one webpage to another by clicking on texts, buttons or images. These clickable elements are called **Hyperlinks**. Hyperlinks are one of the most important features of the World Wide Web because they connect different webpages together.

In HTML, hyperlinks are created using the **Anchor Tag (<a>)**. It allows users to navigate from one webpage to another, open documents, download files, send emails, make phone calls or even jump to different sections of the same webpage.

Without hyperlinks, websites would simply be collections of separate pages with no way to move between them.

<a>... Anchor Tag:

The <a> tag stands for **Anchor Tag**. It is used to create hyperlinks in an HTML document. The clickable text or image is written inside the opening and closing anchor tag works as a link to the webpage or document. The link or location of the file is given inside the **href** attribute.

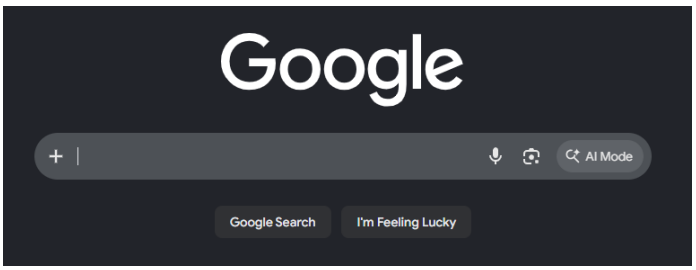
href Attribute:

The **href** attribute stands for **Hypertext Reference**. It specifies the destination or address where the hyperlink will navigate. Without the href attribute, the anchor tag does not know where to go.

E.g.: Click to Visit Google

Output:

Click to Visit Google



Click to Visit Google

The unvisited links are in blue color and the visited links are in purple color. The value of href may contain Website URL, local HTML page, PDF files, images, email addresses etc.

Why <a> Tag is Important?

- **Connects Different Webpages:** It allows users to move from one webpage to another.
- **Improves Website Navigation:** Menus, buttons and navigation bars are all built using anchor tags.
- **Opens External Websites:** Users can visit another website directly.
- **Opens Local HTML Pages:** Different HTML pages inside the same project can be connected together.
- **Creates Internal Navigation:** Users can jump to different sections of the same webpage.

- **Can Open Files:** It can download PDF files, images, Word documents and many other file types.
- **Can Send Emails and Make Phone Calls:** Special links can directly open the email application or phone dialer.

target Attribute:

The **target** attribute specifies where the linked webpage should open when it gets clicked. Normally by default it opens on the same page.

Values of target:

1. target = "_self" (Default value):

Opens the webpage in the same browser tab.

E.g.:

```
<a href="https://www.google.com" target= "_self">Click to Visit Google</a>
```

2. target = "_blank":

Opens the webpage in a new browser tab beside the original tab.

E.g.:

```
<a href="https://www.google.com" target= "_blank">Click to Visit Google</a>
```

3. target = "_parent":

Opens inside the parent frame and it is mostly used with frames.

E.g.:

```
<a href="https://www.google.com" target="_parent">Click to Visit Google</a>
```

4. target = "_top":

Opens in the full browser window. It is also used with frames.

E.g.:

```
<a href="https://www.google.com" target= "_top">Click to Visit Google</a>
```

title Attribute:

The **title** attribute displays a small tooltip whenever the mouse pointer stays over the hyperlink.

E.g.:

```
<a href="https://www.google.com"
title="Open Google">
Click to Visit Google</a>
```

Output:

Internal Hyperlinks:

Sometimes we don't want to move to another webpage. Instead, we want to jump to another section of the same webpage. This is called an Internal Hyperlink. Internal hyperlinks are created using the id attribute. This also uses <a> **Anchor Tag** and the link is created using <a> redirects us to a different section of the same webpage.

Steps to create Internal Hyperlinks:

Step1: Assign an id attribute and value to the destination tag.

```
<h2 id="contact">Contact Us</h2>
```

Step 2: Use the anchor tag. And in **href** give the id name using **hash symbol (#)**.

```
<a href="#contact">Go to Contact Section</a>
```

When clicked, the browser automatically scrolls to the Contact section. Internal Hyperlinks are used for long webpages, table of Contents, FAQs, documentations, Wikipedia pages & portfolio websites.

Types of CSS

Earlier, we discussed about the style attribute and how we can give CSS directly to a tag we want. We mainly use CSS to make webpages beautiful, colorful and responsive. CSS controls colors, fonts, backgrounds, borders, margins, padding, layout, animations, responsive design. We discussed about a few CSS Properties too.

In our webpages the CSS can be added in three different ways;

1. Inline CSS
2. Internal CSS
3. External CSS

Inline CSS:

Inline CSS is written directly inside an HTML element using the **style attribute**. The previously discussed CSS comes under Inline CSS.

E.g.: <h1 style="color:red;">Welcome</h1>

Advantages:

- Easy for beginners.
- For quick testing it's good.

Disadvantages:

- Repeated code.
- Difficult to maintain the HTML and CSS in same area.
- Not suitable for large websites.

Internal CSS:

Internal CSS is written inside the <style> tag. The <style> tag is always written inside the <head> tag section. The CSS properties are exactly the same as we wrote in inline CSS. But here the important thing is selectors. How we select each tag to give the proper CSS Property is the key thing here. There are many types of selectors. But basic selectors are:

- **Element Selector**,
- **Id Selector** and
- **Class Selector**.

E.g.:
```
<head>
    <style>
    h1{
        color:red;
    }
    </style>
</head>
```

Output:

In the example, **Element Selector** is used to give the CSS. Element Selector refers to directly selecting the tag name we want to give CSS to.

Class Selector is often used when there are a specific group of tags require the same type of CSS but we don't want to give CSS to any other similar tags in our HTML File. The class names are given to the specific tags and then it selected inside style tag with the help of (.) dot symbol.

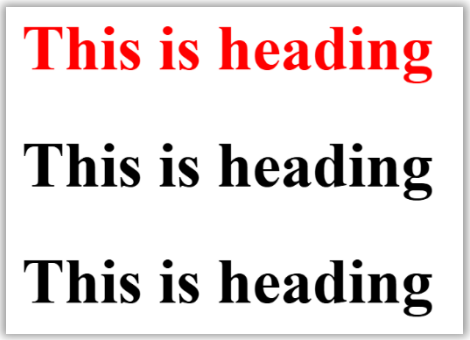
E.g.: <head>
<style>
.heading{
color:red;
}
</style>
</head>
<body>
<h1 class="heading">This is heading</h1>
<h1 class="heading">This is heading</h1>
<h1>This is heading</h1>
</body>

Output:

ID Selector is often used when there is an only one specific and special tag that we want tot give CSS too. Unlike the class, id can not be repeated to another tag, it is only applicable to a single tag. It's mostly used when we try to give a tag an exclusive CSS. It selected inside style tag with the help of (#) hash symbol.

E.g.: <head>
<style>
#heading{
color:red;
}
</style>
</head>
<body>
<h1 id="heading">This is heading</h1>
<h1>This is heading</h1>
<h1>This is heading</h1>
</body>

Output:



There are many more selectors that exist and we will discuss them properly in CSS section of notes.

Advantages:

- Easy for single webpage.
- Better than Inline CSS.
- Styles multiple elements.
- Entire webpage styling stays in one place.
- Reduces repeated code.

Disadvantages:

- Cannot style multiple webpages.
- The CSS length increases and the whole file lines of code also increases.
- HTML file takes both html and css storage.

External CSS:

It is nothing but the same as Internal CSS. We use selectors here and use the same type of CSS properties but the External CSS stores all CSS inside a separate file. The CSS file uses the **.css** extension. It's written outside the html file and we have to establish a connection between the HTML and CSS file to link then each other. For linking the files, we use <link> tag. And its written inside <head> section.

<link> Tag:

The <link> tag connects external resources with an HTML document. Most commonly, it is used to connect CSS files.
E.g.: <link rel="stylesheet" href="style.css">

Here **rel attribute** specifies the relationship of the file that needed to be linked and **href attribute** gives the location or reference of the file to be linked.

Advantages:

- One CSS file can style hundreds of webpages.
- Easy maintenance.
- Faster website development.
- Professional approach.
- Reusable.

Disadvantages:

There are no such disadvantages for this type of CSS as it's used for industry standards.

Difference Between the Three Types of CSS:

Inline CSS	Internal CSS	External CSS
style attribute	style tag	Separate CSS file
Single element	Single webpage	Multiple webpages
Least reusable	Moderate	Most reusable
Not recommended	Small projects	Professional websites

<marquee>...</marquee> Tag:

The <marquee> tag is used to display **scrolling text or images** on a webpage. The content can move left, right, up, down automatically. The marquee tag is widely used for Breaking News, Announcements, Advertisements, Welcome Messages, Offers, Notices etc.
E.g.:

```
<hr>
<marquee>Welcome to LCC Computer Education</marquee>
<hr>
```

Output:

Why Marquee Tag is Important?

- Creates scrolling text.
- Attracts user attention.
- Used for announcements.
- Displays moving texts and images.

Attributes of <marquee>:

direction Attribute:

This attribute controls the direction of movement of texts and images. The text moves toward the direction are given in the value. The default direction is towards left.

E.g.: <marquee direction="right"> Welcome to LCC Computer Education</marquee>

Output:

Welcome to LCC Computer Education 

<marquee direction="up"> Welcome to LCC Computer Education</marquee>

Output:


Welcome to LCC Computer Education

<marquee direction="up"> Welcome to LCC Computer Education</marquee>

Output:

Welcome to LCC Computer Education 

scrollamount Attribute:

This attribute controls the speed of the text or the image that given in the <marquee>. Larger values increase the speed; lower values decrease the speed.

E.g.: <marquee scrollamount="20"> Welcome to LCC Computer Education</marquee>

behavior Attribute:

This attribute controls how the marquee behaves while moving. There are 3 types of behavior values;

- scroll
- slide
- alternate

E.g.: <marquee behavior ="scroll"> Welcome to LCC Computer Education</marquee>

➔ This is the default value, in this value the text moves continuously.

E.g.: <marquee behavior ="slide"> Welcome to LCC Computer Education</marquee>

➔ In this value the text moves once from side and stops at the direction given.

E.g.: <marquee behavior ="alternate"> Welcome to LCC Computer Education</marquee>

➔ In this value the text bounces left and right continuously.

Other attributes like **height, width** works here too also **style** attribute also works to adjust color, font, background etc.

Important Note:

The <marquee> tag is **obsolete (deprecated)** in HTML5. Modern websites achieve scrolling and animation effects using **CSS Animations**, **CSS Transitions**, or **JavaScript** because they provide greater flexibility, better performance, and improved accessibility.

HTML Forms

Whenever we register on a website, log in to Facebook, search something on Google, fill an admission form or order food online, we enter information like our name, email, phone number, password, address etc. This information is collected through HTML Forms.

HTML Forms are one of the most important parts of web development because they allow websites to collect information from users and send it to the server for processing. A form can contain various input controls such as text boxes, radio buttons, checkboxes, drop-down lists, buttons, file upload controls and many more.

<form>...</form> Tag:

An HTML <form> is used to create an HTML form to collect user input. In most browsers forms in fluence the page layout. The start on a new line and have a blank line before and after. The collected user input is most often sent to a server for processing. HTML forms always begin with **<form>** and end with **</form>**.

The <form> tag is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

E.g.:

```
<form>
.
form elements
.
</form>
```

It is a paired tag and itself it is not visible to the webpage. All types of form element are written inside the form tag.

Why HTML Forms are Important?

1. Collects user information
2. Sends data to server
3. Used in Registration Forms
4. Used in Login Forms
5. Used in Search Boxes
6. Used in Contact Forms
7. Used in Online Shopping
8. Holds all Form Elements: It acts as the parent element of all other table tags such as: input, label, textarea, select, option, button etc.

action attribute:

The action attribute of **Form** defines the action to be performed when the form is submitted. Usually, the form data is sent to a file on the backend server when the user clicks on the submit button. And the file source is given as value in this attribute.

In the example below, the form data is sent to a file called "data.php". This file contains a server-side script that handles the form data:

E.g.:

```
<form action="data.php">
    <label for="sname">Student name:</label><br>
    <input type="text" id="sname" name="sname"><br>
    <input type="submit" value="Submit">
</form>
```

method attribute:

The method attribute of **Form** specifies the HTTP method to be used when submitting the form data. The form-data can be sent as URL variables (with method="get") or as HTTP post transaction (with method="post").

The default HTTP method when submitting form data is GET.

E.g. of “get”:

```
<form action="data.php" method="get">
```

Like this we can use the “get” element in method attribute of form tag.

Notes on GET:

- Appends the form data to the URL, in name/value pairs.
- NEVER use GET to send sensitive data! (The submitted form data is visible in the URL!)
- The length of a URL is limited. (2048 characters)
- Useful for form submissions where a user wants to bookmark the result.
- GET is good for non-secure data, like query strings in Google.

E.g. of “post”:

```
<form action="data.php" method="post">
```

Like this we can use the “get” element in method attribute of form tag.

Notes on POST:

- Appends the form data inside the body of the HTTP request. (The submitted form data is not shown in the URL)
- POST has no size limitations, and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked.

Difference Between **GET** and **POST**

GET	POST
Data visible in URL	Hidden from URL
Bookmark possible	Bookmark not possible
Small data	Large data
Less secure	More secure

The HTML <form> Elements:

The HTML <form> element can contain one or more of the following form elements: <label>, <input>, <select>, <textarea>, <button>, <fieldset>, <legend>, <datalist>, <option>.... etc.

The <label> Element:

The <label> element defines a label for several form elements. It is useful for screen-reader users. It also helps users who have difficulty clicking on very small regions (such as radio buttons or checkboxes) - because when the user clicks the text within the <label> element, it toggles the radio button/checkbox.

The **for** attribute of the <label> tag should be equal to the **id** and **name** attribute of the <input> element to bind them together.

E.g.:

```
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname">
```

Output:

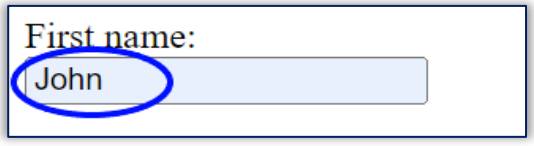
The <input> Element:

One of the most used form elements is the <input> element. The <input> element can be displayed in several ways, depending on the type attribute. There are several type attributes in input tags such as “text”, “date”, “email”, “password”, “number” etc. It’s an unpaired tag.

E.g.:

```
<form action="data.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname">
</form>
```

Output:



It was an example of **<input type= “text”>** and by default value of **type** attribute is set to **text**. The whole form is consists of input tags. And it also consists some important attributes like id, name, placeholder, value, size.

id Attribute:

The **id** attribute is used to give a **unique identification (ID)** to an HTML element. Every element should have a different id value because it uniquely identifies that particular element. It is mainly used to connect the <label> tag with the <input> tag and can also be used by CSS and JavaScript as well as for the backend data.

E.g.: <input type="text" id="fname">

Note: The value of the **id** attribute should always be unique within the webpage. Also, the **for** attribute of the <label> tag should be the same as the **id** attribute of the <input> tag to bind them together.

name Attribute:

The **name** attribute is used to give a **name** to an input field. When a form is submitted, the data entered by the user is sent to the server using the value of the **name** attribute. It is one of the most important attributes of the <input> tag. It also helps to bind the <label> tag.

E.g.: <input type="text" id="fname" name="fname">

Note: In most cases the **id** and **name** attributes values are kept same.

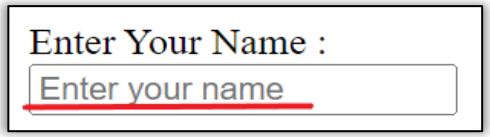
placeholder Attribute:

The **placeholder** attribute is used to display a **short indicator or faded text** inside an input field before the user enters any value. It helps the user understand what type of information should be entered.

E.g.:

```
<input type="text"
placeholder="Enter Your Name">
```

Output:



Note: The placeholder text disappears automatically when the user starts typing inside the input field. It should only be used as a hint and not as a replacement for the <label> element.

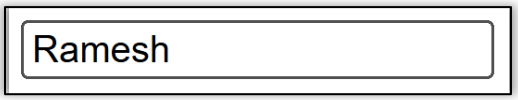
value Attribute:

The **value** attribute is used to specify the **default value** of an input field. The given value automatically appears inside the input field when the webpage loads.

E.g.:

```
<input type="text" value="Ramesh">
```

Output:



Note: The value attribute is also used to change the text displayed on buttons such as **Submit**, **Reset**, and **Button**.

size Attribute:

The **size** attribute is used to specify the **visible width** of an input field. The value represents the approximate number of characters that can be displayed inside the input box.

E.g.:

```
<input type="text" placeholder="Enter Your Name" size="30">
```

Output:

Various <input> tag attributes:

Input type Text:

<input type="text"> defines a single-line text input field. As we saw in the previous example. We can change its design and look with the help of some attributes. In this type of field, we can enter any type of data which includes lowercase and uppercase letters, digits, numbers etc.

E.g.:

```
<input type= “text”>
```

Output:

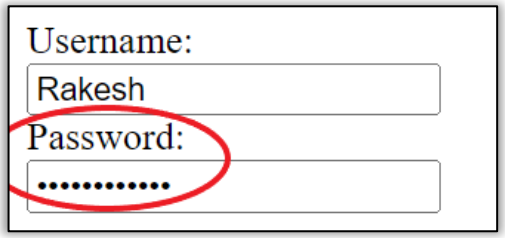
Input Type Password:

<input type="password"> defines a password field.

Output:

E.g.:

```
<label for="username">Username:</label><br>
<input type="text" id="username"><br>
<label for="password">Password:</label><br>
<input type="password" id="password">
```



Here as we enter any text in the password section, the text automatically hides and converts to hidden dots. we also can enter any type of data which includes lowercase and uppercase letters, digits, numbers etc in this field. We can add the unhide feature of the password using only JavaScript.

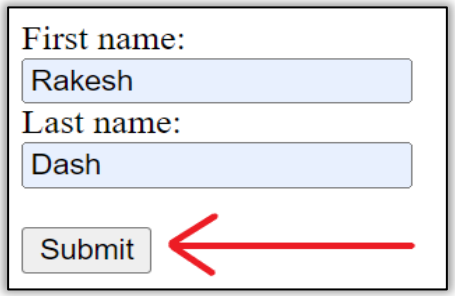
Input Type Submit:

<input type="submit"> defines a button for submitting the form data to a form-handler. The form-handler is typic ally a server page with a script for processing input data. It stores the data we entered in the form.

E.g.:

```
<label for="fname">First name:</label><br>
<input type="text" id="fname"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname"><br><br>
<input type="submit">
```

Output:



It creates a **Submit** button underneath the above text to submit the form data.

Input Type Reset:

<input type="reset"> defines a reset button that will reset all form values to their default values.

E.g.:

```
<input type="submit">
<input type="reset">
```

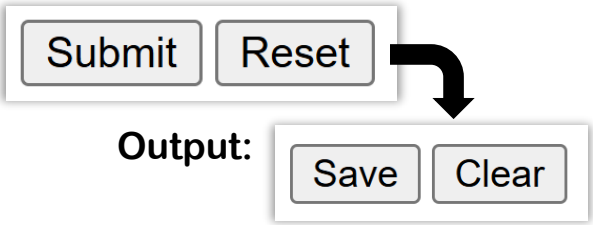
Output:

It erases all the data inputted previously in the form. But if there is a data written in value attribute then it will not erase it, the data that value attribute contains will still be in the text field.

Note: With the help of value attribute, we can change the button display text of the submit and reset button. Their work functions will remain same but the text on the button only be changed.

E.g.:

```
<input type="submit" value="Save">
<input type="reset" value="Clear">
```



Input Type Radio:

This tag defines a **radio button**. Radio buttons are a set of options in which we can select **only one option** from all the displayed options in the list. They are generally used when only one choice is allowed, such as **Gender, Category, Payment Method, Marital Status** etc.

E.g.:

```
<label>Gender:</label>
<input type="radio" id="male" name="gender" value="male">
<label for="male">Male</label>
<input type="radio" id="female" name="gender" value="female">
<label for="female">Female</label>
<input type="radio" id="other" name="gender" value="other">
<label for="other">Other</label>
```

Output:

Note: The **for** attribute of the <label> tag should be the same as the **id** attribute of the <input> tag to bind them together. Also, all radio buttons should have the **same name attribute** so that only one option can be selected at a time. The **value** attribute specifies the value that will be sent to the server when that option is selected.

Input Type Checkbox:

This tag defines a **checkbox**. Checkboxes are a set of options in which we can select **one option, multiple options, or even all the available options** depending upon the requirement. They are commonly used for selecting **Hobbies, Skills, Qualifications, Interests** etc.

E.g.:

```
<label>Education:< /label>
<input type="checkbox" id="10th" name="education" value="10th">
<label for="10th">10th</label>
<input type="checkbox" id="+2" name="education" value="+2">
<label for="+2">+2</label>
<input type="checkbox" id="+3" name="education" value="+3">
<label for="+3">+3</label>
```

Output:

Note: The **for** attribute of the <label> tag should be the same as the **id** attribute of the <input> tag to bind them together. Unlike radio buttons, checkboxes **do not require the same name attribute to limit the selection**, therefore users can select **multiple options** at the same time. The **value** attribute specifies the value that will be sent to the server for each selected checkbox.

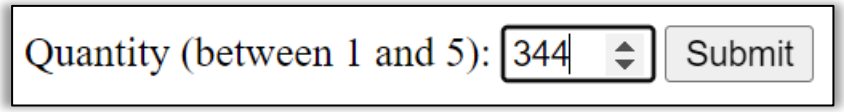
Input Type Number:

This tag defines a **numeric input field**. It is used to accept only numeric values from the user. Normally, letters and most special characters cannot be entered in this field. However, depending on the browser, it may also allow symbols like **'+' (plus), '-' (minus), '.' (decimal point)** and the letter **'e'** (used for scientific notation). We can also set the minimum and maximum range of numbers by using the **min** and **max** attributes. If the entered value does not meet the specified conditions, the browser will display a validation error when the form is submitted.

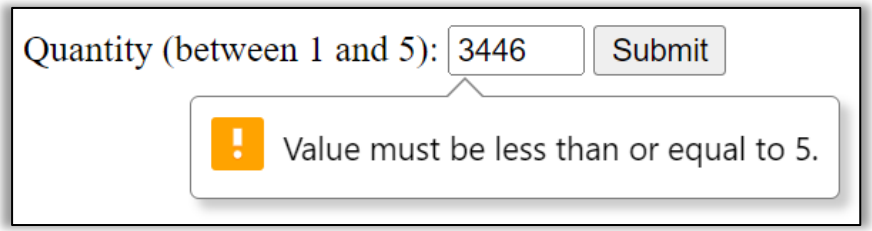
E.g.:

```
<label for="quantity">Quantity (between 1 and 5):</label>
<input type="number" id="quantity" name="quantity" min="1" max="5">
<input type="submit" value="Submit">
```

Output:



From the above example, we can only enter numbers between **1 and 5**. If we enter a value less than **1** or greater than **5**, the browser will display a validation error like the one shown below.



Note: By default, an **up arrow** and a **down arrow** (also known as **spinner controls**) appear beside the number input field. These arrows can be used to increase or decrease the value one step at a time. We can also control the increment or decrement value by using the **step** attribute.

Input Type Email:

<input type="email"> is used for input fields that should contain an **e-mail address**. The browser automatically checks whether the entered text follows the correct e-mail format (such as username@example.com). If the entered value is not a valid e-mail address, the browser displays a validation error when the form is submitted.

E.g.:

```
<label for="email">Enter your email:</label>
<input type="email" id="email" name="email">
<input type="submit" value="Submit">
```

Output:

Note: If we enter a normal text instead of a valid e-mail address, the browser will display a validation error while submitting the form. This validation helps ensure that users enter a properly formatted e-mail address.

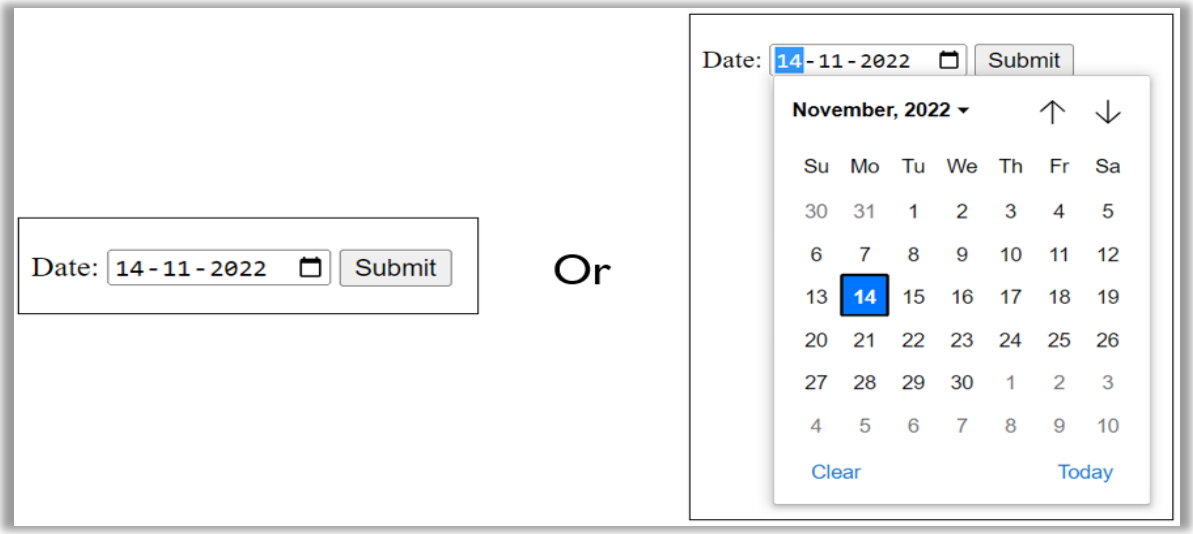
Input Type Date:

<input type="date"> is used for input fields that should contain a **date**. Depending on browser support, a **date picker (calendar)** appears beside the input field, allowing users to select a date easily. Users can either type the date manually or choose it from the calendar. The appearance of the date picker may vary from one browser to another.

E.g.:

```
<label for="dob">Date:</label>
<input type="date" id="dob" name="dob">
<input type="submit" value="Submit">
```

Output:



Note: The date can be entered either by manually typing it or by selecting it from the date picker as shown in the picture. The design of the date picker is provided by the browser and can be customized using CSS only to a limited extent.

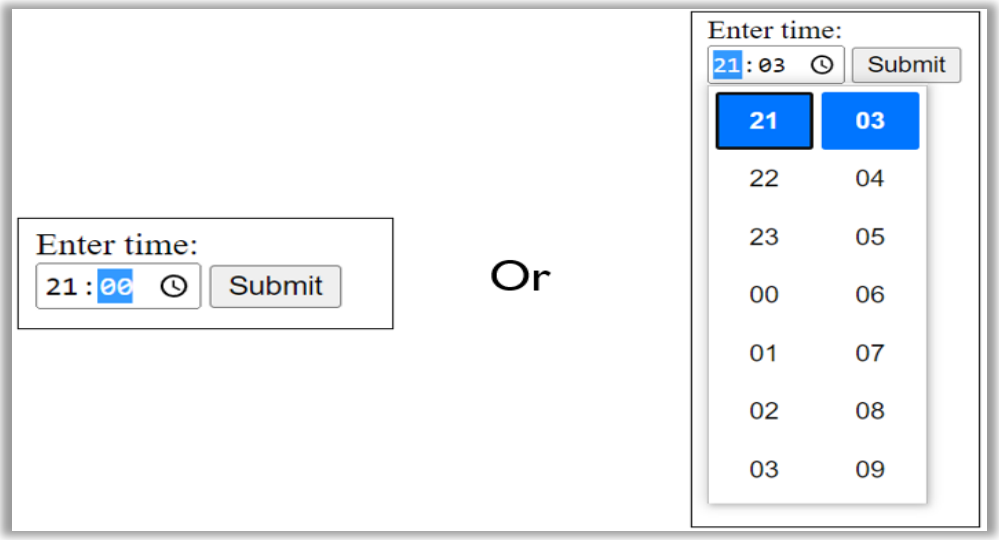
Input Type Time:

<input type="time"> is used to allow the user to select a **time**. Depending on browser support, a **time picker** appears beside the input field. Users can either type the time manually or select it using the time picker.

E.g.:

```
<label for="time">Enter Time:</label><br>
<input type="time" id="time" name="time">
<input type="submit" name="submit">
```

Output:



Note: The time can be entered either by manually typing it or by selecting it from the time picker. Similar to the **Date** input type, the appearance of the time picker may vary depending on the browser.

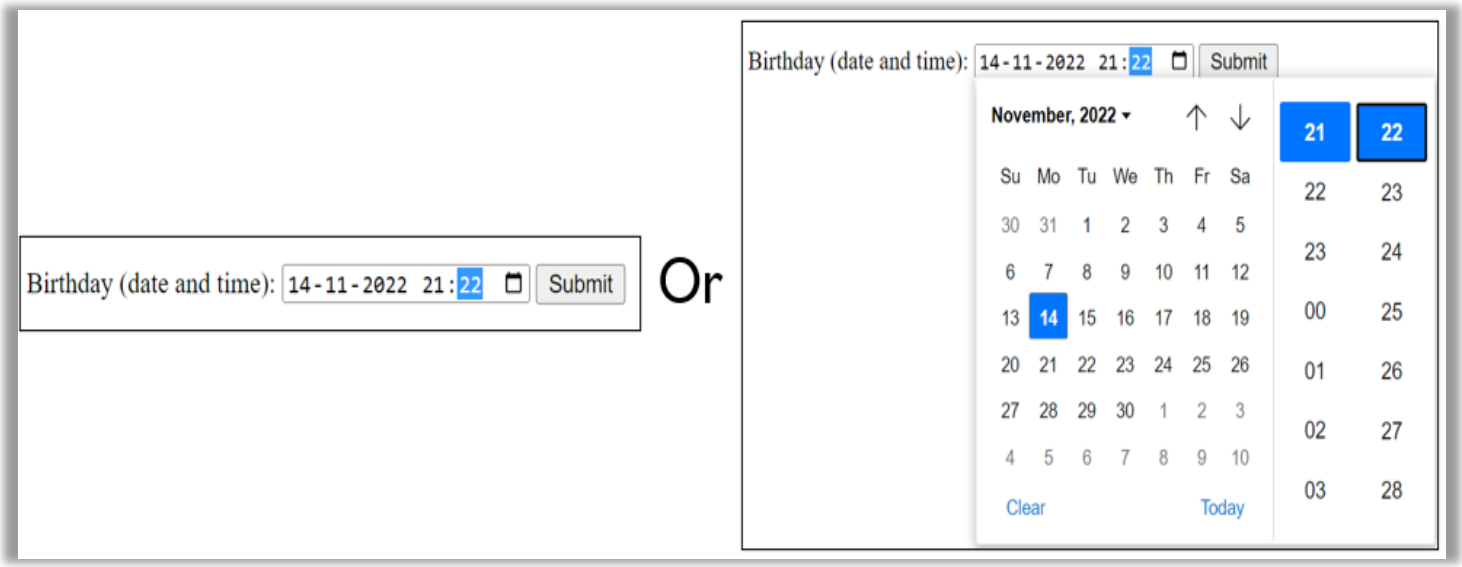
Input Type Datetime-Local:

<input type="datetime-local"> specifies a date and time input field. It is a combination of both the Date and Time input types. Depending on browser support, a date and time picker appears beside the input field, allowing users to select both the date and time together.

E.g.:

```
<label for="birthdaytime">Birthday (Date and Time):</label>
<input type="datetime-local" id="birthdaytime" name="birthdaytime">
<input type="submit" value="Submit">
```

Output:



Note: The date and time can be entered either by manually typing them or by selecting them from the date-time picker. The appearance of the picker depends on the browser being used.

The <textarea> Element:

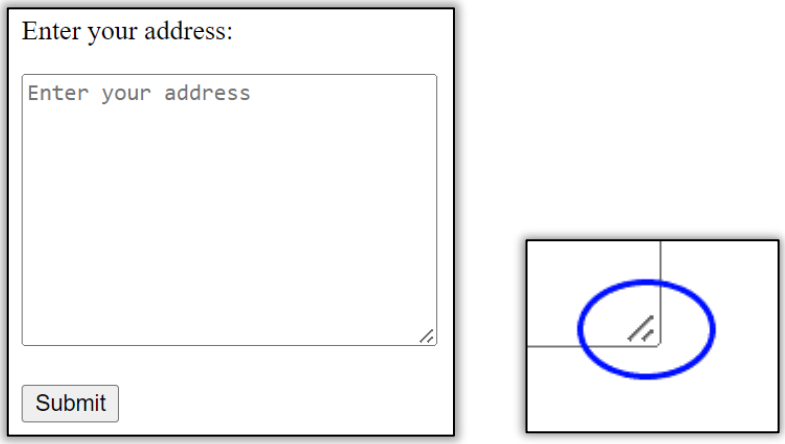
The <textarea> element defines a **multi-line input field** or a **large text area**. It is used to accept long pieces of text such as **addresses, comments, feedback, suggestions, descriptions, biography (Bio)** etc. Unlike the <input> element, the <textarea> element has both an opening tag and a closing tag.

The **rows** attribute specifies the visible number of lines in the text area, whereas the **cols** attribute specifies the visible width of the text area. By default, **rows="10"** and **cols="30"**. We can change the values of **rows** and **cols** according to our requirements. We can also use the **placeholder** attribute to display a faded indicator inside the text area before the user starts typing.

E.g.:

```
<textarea name="message" rows="10" cols="30" placeholder="Enter your address"></textarea>
<br><br>
<input type="submit">
```

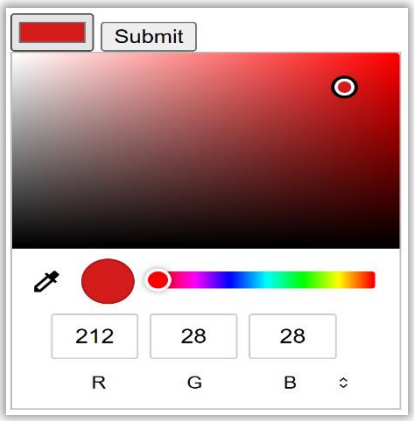
Output:



Note: This is the default size of the <textarea> element. We can increase or decrease its size by changing the values of the **rows** and **cols** attributes. By default, users can also resize the text area by dragging the **small handle** available at the bottom-right corner of the text area.

Input Type Color:

This tag defines a **color picker** that allows the user to select a color. Depending on browser support, clicking on the input field opens a color selection dialog from which users can choose any color.



E.g.:

```
<label for="favcolor">Choose your favourite color:
</label>
<input type="color" id="favcolor" name="favcolor">
<input type="submit" value="Submit">
```

Output:

Note: After selecting a color, its hexadecimal color code (such as **#ff0000**) is automatically stored as the value of the input field.

Input Type Range:

This tag defines a **slider control** that allows users to select a numeric value within a specified range. It is commonly used for selecting **volume, brightness, rating, temperature** and similar values. The **min** attribute specifies the minimum value, the **max** attribute specifies the maximum value and the **step** attribute specifies the interval between values.

E.g.:

```
<label for="volume">Volume:</label>
<input type="range" id="volume" name="volume" min="0" max="100" step="10">
<input type="submit" value="Submit">
```

Output:

Note: The slider can be moved left or right to select a value within the specified range. If the **step** attribute is not specified, the default step value is **1**.

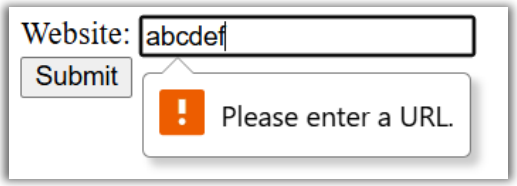
Input Type URL:

This tag is used for input fields that should contain a **website address (URL)**. The browser checks whether the entered text is in a valid URL format before submitting the form.

E.g.:

```
<label for="website">Website:</label>
<input type="url" id="website" name="website">
<input type="submit" value="Submit">
```

Output:



Note: If the entered text is not a valid URL (such as https://www.example.com), the browser displays a validation error while submitting the form.

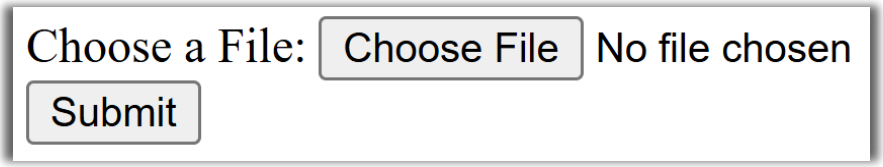
Input Type File:

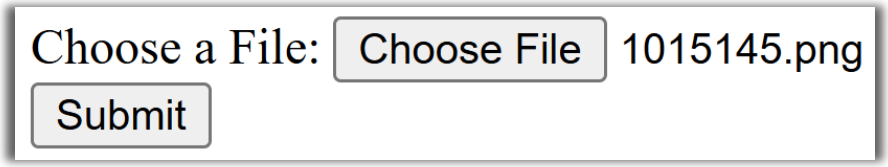
<input type="file"> defines a **file upload field** that allows users to select one or more files from their computer or mobile device. It is commonly used to upload **images, documents, videos, PDFs** and other files.

E.g.:

```
<label for="photo">Choose a File:</label>
<input type="file" id="photo" name="photo">
<input type="submit" value="Submit">
```

Output:





Note: Clicking the **Choose File** (or **browse**) button opens the device's file explorer, allowing users to select a file. We can also allow users to upload multiple files at once by using the **multiple** attributes.

The <select> and <option> Element:

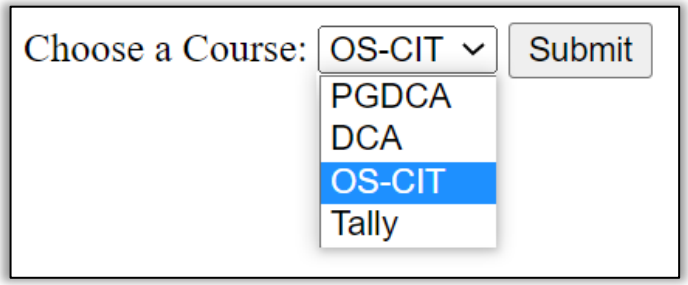
The <select> element defines a **drop-down list** that allows the user to choose **one option** from a list of available options. The <option> element defines each individual option inside the drop-down list. The <select> tag and the <option> tag are always used together.

The **value** attribute of the <option> element specifies the value that will be sent to the server when that option is selected.

E.g.:

```
<label for="course">Choose a Course:</label>
<select id="course" name="course">
   <option value="PGDCA">PGDCA</option>
   <option value="DCA">DCA</option>
   <option value="OS-CIT">OS-CIT</option>
   <option value="Tally">Tally</option>
</select>
<input type="submit">
```

Output:



Note: The <select> tag creates the drop-down list, while the <option> tag defines all the options displayed inside it. All <option> elements must be written inside the <select> tag; otherwise, the drop-down list will not work properly. By default, the first option is automatically selected unless another option is specified using the **selected** attribute.

The <datalist> Element:

The <datalist> element specifies a **list of pre-defined options** for an <input> element. As the user starts typing inside the input field, the browser automatically displays matching suggestions from the predefined list. Unlike the <select> element, users can either choose one of the suggested options or enter their own custom value.

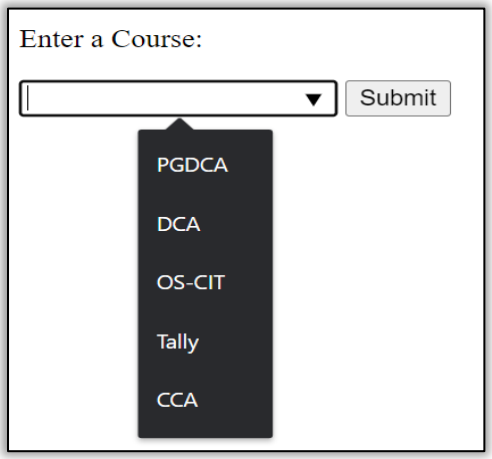
The <datalist> element uses the <option> element to define the list of suggestions.

E.g.:

```
<p>Enter a Course:</p>
<input list="course" name="course">
```

```
<datalist id="course">
   <option value="PGDCA">
   <option value="DCA">
   <option value="OS-CIT">
   <option value="Tally">
   <option value="CCA">
</datalist>
<input type="submit">
```

Output:



Note: The **list** attribute of the <input> element must refer to the **id** attribute of the <datalist> element. Otherwise, the suggestions will not appear. Unlike the <select> element, users can also enter values that are not available in the suggestion list.

Difference between <select> and <datalist>:

<select>	<datalist>
Creates a drop-down list.	Creates an auto-suggestion list.
Users can select only from the given options.	Users can either select a suggested option or type their own value.
Uses <select> and <option> tags together.	Uses <input>, <datalist> and <option> tags together.
Commonly used for fixed choices such as Gender, Country, Course etc.	Commonly used for search suggestions, city names, products, browser suggestions etc.

The <fieldset> and <legend> Element:

The <fieldset> element is used to **group related form elements** together inside a form. It creates a **bordered box** around all the form elements written between the opening and closing <fieldset> tags, making the form more organized and easier to understand.

The <legend> element is used to define a **caption or heading** for the <fieldset> element. The text written inside the <legend> tag appears at the top of the bordered box.

E.g.:

```
<fieldset>
   <legend>Information:</legend>
   <label for="fname">First Name:</label><br>
   <input type="text" id="fname"><br>
   <label for="lname">Last Name:</label><br>
   <input type="text" id="lname"><br><br>
   <input type="submit" value="Submit">
</fieldset>
```

Output:

Note: The <fieldset> element groups related form controls together by displaying them inside a bordered box, whereas the <legend> element provides a title or heading for that group. Using <fieldset> and <legend> makes forms more organized, readable and user-friendly, especially when the form contains multiple sections.

Some other important attributes:

required Attribute:

The required attribute is used to make an input field mandatory. It ensures that the user cannot submit the form without filling in the required field. If the user leaves the field empty and tries to submit the form, the browser automatically displays a validation error.

The required attribute can be used with various form elements such as text, password, email, number, date, textarea, select, file and many other input types.

E.g.:

```
<label for="name">Name:</label>
<input type="text" id="name"
name="name" required><br><br>
<input type="submit" value="Submit">
```

Output:

Note: The required attribute is a boolean attribute, which means it does not require any value. Simply writing required inside the tag makes the field mandatory. It is commonly used in Registration Forms, Login Forms, Contact Forms and other forms to ensure that important information is not left empty before submission.

The readonly Attribute:

The **readonly** attribute is used to make an input field **read-only**. A read-only field can display a value, but the user **cannot modify or edit** it. However, the value of the read-only field is still submitted along with the form data. The **readonly** attribute is commonly used when information should be displayed to the user but should not be changed, such as **User ID, Registration Number, Username** etc.

E.g.:

```
<label for="userid">User ID:</label>
```

```
<input type="text" id="userid" name="userid" value="LCC101" readonly>
<br><br><input type="submit" value="Submit">
```

The disabled Attribute:

The **disabled** attribute is used to **disable** an input field. A disabled field cannot be clicked, focused, edited or selected by the user. It usually appears faded or greyed out in the browser.

The **disabled** attribute is commonly used when an input field is temporarily unavailable or should not be used until certain conditions are met.

E.g.:

```
<label for="city">City:</label>
<input type="text" id="city" name="city" value="Bhubaneswar" disabled>
```

Output:

Note: The value of a disabled input field is **not submitted** to the server when the form is submitted.

HTML Block Level Elements and Inline Elements

Whenever we create a webpage, different HTML elements behave differently on the browser. Some elements automatically start from a **new line** and occupy the entire available width of the webpage, while others occupy only the required amount of space and continue on the same line.

Based on this behaviour, HTML elements are mainly divided into **two types**:

1. **Block Level Elements**
2. **Inline Elements**

Understanding the difference between these two types of elements is very important because it helps us design and organize webpages properly.

Why is it Important?

Understanding Block Level and Inline Elements helps us:

- Arrange webpage content properly.
- Create well-structured layouts.
- Improve the readability of webpages.
- Apply CSS more effectively.
- Build responsive websites more easily.

Almost every webpage uses both Block Level and Inline Elements together.

Block Level Elements:

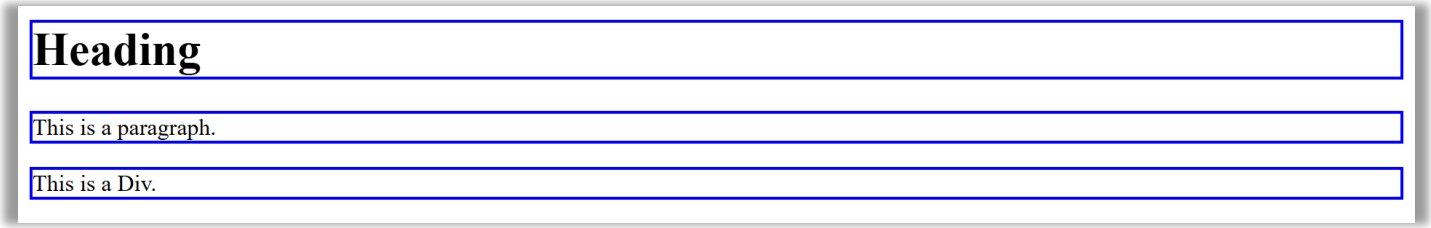
A **Block Level Element** is an HTML element that always starts from a **new line** and occupies the **full available width** of its parent container by default. Even if the content inside the element is very small, it still takes the complete width available.

For this reason, every Block Level Element appears one below another.

E.g.:

```
<h1>Heading</h1>
```

This is a paragraph.
This is a Div.
Output:



As we can see, the border we have given shows every element starts from a **new line** automatically.

How Block Level Elements Work?

Suppose the browser window is **1000 pixels wide**. Even if we write Hello ;

The word **Hello** occupies only a few pixels, but the element itself occupies the **entire width** of the browser. That is why the next element always starts from the next line.

Features of Block Level Elements

- Always start from a new line.
- Occupy the full available width by default.
- Can contain both Block Level Elements and Inline Elements.
- Used for creating the basic structure and layout of a webpage.
- Width and height can be easily controlled using CSS.

Examples of Block Level Elements

Some commonly used Block Level Elements are: <h1> to <h6>, , , <table>, <form>, <ul>, <ol>, <li>, <fieldset>

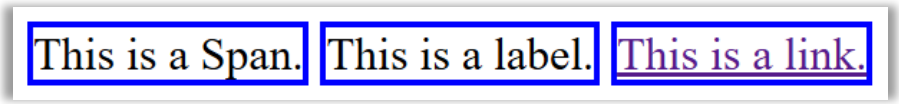
Inline Elements:

An **Inline Element** is an HTML element that occupies **only the amount of space required by its content**. Unlike Block Level Elements, Inline Elements **do not start from a new line**. They continue on the same line until there is no space left.

Inline Elements are generally used to format or modify a small part of the content inside Block Level Elements.

E.g.:

This is a Span.
<label>This is a label.</label>
This is a link.
Output:



In these different tags is only taking the space of the content only, and do not occupy the whole line.

How Inline Elements Work?

Suppose we write

```
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Contact</a>
```

All three hyperlinks appear on the **same line** because the <a> tag is an Inline Element. If there is no remaining space, the browser automatically moves the next Inline Element to the following line.

Features of Inline Elements

- Do not start from a new line.
- Occupy only the required width.
- Continue on the same line.
- Mainly used for formatting text or small portions of content.
- Width and height generally cannot be applied directly in the same way as block elements (without changing their display property).

Examples of Inline Elements

Some commonly used Inline Elements are: , <a>, <img>, , <b>, <i>, , <u>, <label>, <input>, <button>

Difference Between Block Level and Inline Elements:

Block Level Elements	Inline Elements
Starts from a new line.	Does not start from a new line.
Occupies the full available width.	Occupies only the required width.
Appears one below another.	Appears on the same line.
Used for webpage structure and layout.	Used for formatting small portions of content.
Can contain both Block Level and Inline Elements.	Generally, contains text or other Inline Elements.
Example: , <p>, <table>	Example: , <a>, <img>

HTML5 Semantic Elements

In earlier versions of HTML, developers mostly used the tag to create different sections of a webpage. Since every section was created using the same tag, it was difficult for browsers, search engines and screen readers to understand the purpose of each section.

To solve this problem, **HTML5 introduced Semantic Elements**. Semantic elements clearly describe the purpose of the content they contain, making webpages more meaningful and well-structured. The word **Semantic** means **"having meaning."** Therefore, Semantic Elements are HTML tags whose names clearly describe their purpose.

Why were Semantic Elements Introduced?

Semantic elements were introduced in HTML5 to:

- Make HTML code easier to read and understand.

- Improve the structure of webpages.
- Help search engines understand webpage content (SEO).
- Improve accessibility for screen readers.
- Reduce the excessive use of tags.
- Make websites easier to maintain.

Common HTML5 Semantic Elements:

Some commonly used semantic elements are:

- <header> – Defines the header section of a webpage.
- <nav> – Defines a navigation bar of a webpage.
- <section> – Defines a separate section of content.
- <article> – Defines an independent article or blog post.
- <aside> – Defines content related to the main content, such as sidebars.
- <main> – Defines the main content of the webpage.
- <footer> – Defines the footer section of a webpage.

E.g.:

```
<header>
  Website Header
</header>

<nav>
  Home | About | Contact
</nav>

<main>
      <section>
            <h2>HTML Notes</h2>
            <p>This is a section. </p>
      </section>
      <article>
            <h2>Latest News</h2>
            <p>This is an article. </p>
      </article>
</main>

<footer>
  Copyright © 2026
</footer>
```

Note: Semantic elements do not change the appearance of a webpage by themselves. They simply provide meaningful structure to the content. The design and styling of these elements are done using CSS.

MULTIMEDIA TAGS

The <audio> Element:

The <audio> element is used to embed and play **audio files** directly on a webpage without using external plugins. The <source> element is used inside the <audio> tag to specify the location and format of the audio file.

E.g.:

```
<audio controls>
   <source src="music.mp3">
</audio>
```

Output:

Important Attributes of <audio>

controls Attribute:

Displays the default audio controls such as **Play, Pause, Volume and Progress Bar**.

E.g.: <audio controls>

autoplay Attribute:

Automatically starts playing the audio when the webpage loads.

E.g.: <audio autoplay>

loop Attribute:

Plays the audio repeatedly after it ends.

E.g.: <audio loop>

muted Attribute:

Starts the audio in muted mode.

E.g.: <audio muted>

Note: Multiple <source> elements can be used inside the <audio> tag to support different audio formats such as **MP3, OGG and WAV**.

The <video> Element:

The <video> element is used to embed and play **video files** directly on a webpage without using external plugins. The <source> element specifies the video file to be played.

E.g.:

```
<video width="500" controls>
   <source src="video.mp4">
</video>
```

Output:

Important Attributes of <video>

controls Attribute:

Displays the default video controls such as **Play, Pause, Volume, Progress Bar and Full Screen**.

E.g.: <video controls>

width Attribute:
Specifies the width of the video player.
E.g.: <video width="500">

height Attribute:
Specifies the height of the video player.
E.g.: <video height="300">

autoplay Attribute:
Automatically starts playing the video when the webpage loads.
E.g.: <video autoplay>

loop Attribute:
Continuously plays the video after it finishes.
E.g.: <video loop>

muted Attribute:
Starts the video without sound.
E.g.: <video muted>

poster Attribute:
Displays an image before the video starts playing.
E.g.: <video poster="thumbnail.jpg">

Note: Multiple <source> elements can also be used inside the <video> tag to support different video formats such as **MP4, WebM and OGG**.

The <iframe> Element:

The <iframe> (Inline Frame) element is used to **embed another webpage or online content** inside the current webpage. It allows us to display content such as **YouTube videos, Google Maps, PDF files, websites** and many other online resources without leaving the current page.

E.g.:
```
<iframe
src="https://www.wikipedia.org"
width="600"
height="400">
</iframe>
```

Output:

Important Attributes of <iframe>

src Attribute:
Specifies the URL or webpage that should be displayed inside the iframe.
E.g.:<iframe src="https://www.google.com"></iframe>

width Attribute:
Specifies the width of the iframe.
E.g.:<iframe width="600"></iframe>

height Attribute:
Specifies the height of the iframe.
E.g.:<iframe height="400"></iframe>

title Attribute:
Provides a title for the iframe to improve accessibility and help screen readers identify its purpose.
E.g.:<iframe title="Google Map"></iframe>

allowfullscreen Attribute:
Allows the embedded content, such as a YouTube video, to be viewed in full-screen mode.
E.g.:<iframe allowfullscreen></iframe>

loading Attribute:
Specifies when the iframe should be loaded.
Values:

- lazy
- eager

E.g.:<iframe loading="lazy"></iframe>

Note: Some websites do not allow themselves to be displayed inside an iframe for security reasons. However, services like **YouTube**, **Google Maps**, and many online documents provide special embed links that work perfectly with the <iframe> element.

CSS Selectors

After learning HTML, we know how to create the **structure** of a webpage by adding headings, paragraphs, images, tables, forms and many other elements. However, these HTML elements appear with the browser's default styling and may not look attractive.

To apply different styles such as **colors, fonts, backgrounds, borders, spacing and layouts**, we use **CSS (Cascading Style Sheets)**. But before applying any style, CSS must know **which HTML element** should receive that style.

To identify and select HTML elements, CSS uses **Selectors**.A **CSS Selector** is a pattern used to select one or more HTML elements on which CSS properties will be applied. Selectors are one of the most important concepts in CSS because every CSS rule begins with a selector.

Why are CSS Selectors Important?

CSS Selectors are important because they:

- Select one or more HTML elements for styling.
- Help apply different styles to different elements.
- Reduce repeated code.
- Make webpages more organized and easier to maintain.
- Allow developers to target elements in different ways according to their requirements.

Without CSS Selectors, it would not be possible to apply styles to specific HTML elements.

General Syntax of a CSS Selector

Every CSS rule follows a simple syntax.

```
selector{
        property: value;
}
```

E.g.:

```
p{
        color: red;
}
```

Here,

- **p** → Selector
- **color** → Property
- **red** → Value

This CSS rule changes the text color of all paragraph elements to red.

Types of CSS Selectors

CSS provides different types of selectors for selecting HTML elements in various ways. The most commonly used selectors are:

- Universal Selector
- Element Selector

- ID Selector
- Class Selector
- Descendant Selector
- Child Selector
- Pseudo Class Selectors
- Pseudo Element Selectors

Let us understand each selector one by one.

Universal Selector (*):

The **Universal Selector** is represented using the **asterisk (*)** symbol.
It selects **every HTML element** present on the webpage and applies the specified CSS properties to all of them.

Syntax:

```
*{
   property:value;
}
```

E.g.:

```
*{
   color:blue;
}
```

Every text element on the webpage becomes blue.

Why is Universal Selector Important?

- Selects every HTML element.
- Removes browser default styling.
- Commonly used for CSS Reset.
- Saves time by applying common styles to all elements.

Real-life Example:

Almost every professional website's CSS starts with:

```
*{
   margin:0;
   padding:0;
   box-sizing: border-box;
}
```

This removes the browser's default spacing and makes webpage layouts more consistent across different browsers.

Note: Since the Universal Selector affects every HTML element, it should be used carefully. Applying too many properties using this selector may affect the performance and appearance of the webpage.

Element Selector:

The **Element Selector** is also known as the **Tag Selector**. It selects HTML elements based on their tag names.

If multiple elements have the same tag name, all of them will receive the specified style.

Syntax

```
tagname{
    property:value;
}
```

E.g.:

```
p{
    color:red;
}
```

Every paragraph becomes red.

Output:

Why is Element Selector Important?

- Styles all elements of the same type.
- Easy to understand.
- Best for common styling.

Note: If there are ten <p> tags on the webpage, all ten paragraphs will receive the same style.

Class Selector (.):

The **Class Selector** is used to style one or more HTML elements using their **class** attribute. It's like giving another name to the existing tags to make it stand out from the other tags. It is represented using a **dot (.)** followed by the class name in CSS. The same class can be assigned to multiple elements so there can be multiple tags with same class name.

Syntax

```
.classname{
    property: value;
}
```

E.g.:

HTML:

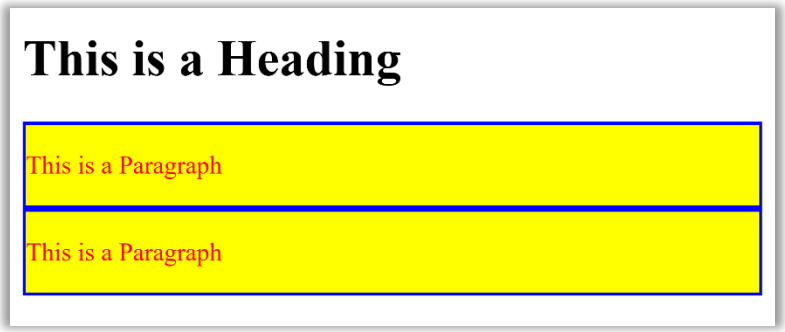
```
<div>
<h1>This is a Heading</h1>
</div>
<div class="text-box">
<p>This is a Paragraph</p>
</div>
<div class="text-box">
<p>This is a Paragraph</p>
</div>
```

CSS:

```
.text-box{
    color: red;
    background-color: yellow;
    border: 2px solid blue;
}
```

Both elements become text boxes with the given CSS.

Output:



Why is Class Selector Important?

- Styles multiple elements together.
- Reusable CSS in other tags.
- Most commonly used selector in modern web development.

Note: One HTML element can have multiple class names, and the multiple class names should be written in same attribute differentiating with spaces. If in a case we have to write a big class name with multiple word then we need to write it in a single word like in the example given above.

ID Selector (#):

The **ID Selector** is used to style a specific HTML element using its **id** attribute. It is represented using the **hash (#)** symbol followed by the ID name. Unlike Class attribute, the value of an **id** should always be unique so an ID Selector usually styles only one element.

Syntax

```
#idname{
    property:value;
}
```

E.g.: Taking the example of previous class code and including ID into it.

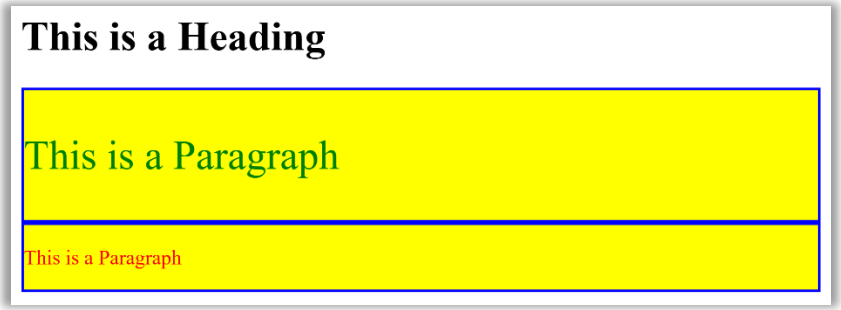
HTML:

```
<h1>This is a Heading</h1>
</div>
<div class="text-box" id="box1">
<p>This is a Paragraph</p>
</div>
<div class="text-box">
<p>This is a Paragraph</p>
</div>
```

CSS:

```
#box1{
    color: green;
    font-size: 32px;
}
```

Output:



Why is ID Selector Important?

- Selects only one specific element.
- Used when unique styling is required.
- Faster and more specific than Element Selector.

Note: An **id** should be unique within a webpage. Multiple elements should not have the same ID value. You can also take multiple id names into one element but it not preferred much. The id name should be unique because it deals with backend data and form related linking. That is why ID is always should be one for each tag.

Difference between ID Selector and Class Selector

ID Selector	Class Selector
Uses id attribute.	Uses class attribute.
Starts with (#) in CSS.	Starts with (.) in CSS.
Should be unique.	Can be reused.
Styles one element.	Styles multiple elements.

Child Selectors

Direct Child Selector (>)

The **Child Selector** selects only the **direct child elements** of a parent element. It is represented using the greater-than symbol (**>**).

Syntax

```
parent > child {
    property: value;
}
```

Here the parent should always be in left side and the child should always be in right side.

E.g.:

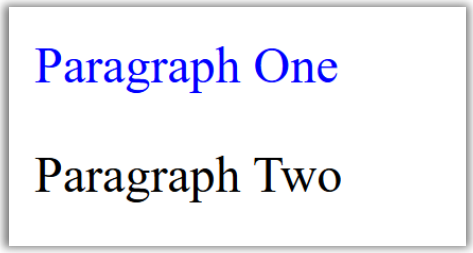
HTML:

```
<div>
<p>Paragraph One</p>
<section>
<p>Paragraph Two</p>
</section>
</div>
```

CSS:

```
div > p {
    color: blue;
}
```

Output:



Only **Paragraph One** becomes blue because it is the direct child of .

Note: Always the parent needs to be in the left side of the > symbol and the child should be at right side. We can also use class or id names instead of direct element names.

Descendant Child Selector (Space)

The **Descendant Child Selector** selects all matching child elements that are present anywhere inside a parent element, whether they are direct children or nested deeper. It is represented using a **space** between two selectors.

Syntax

```
parent child {
    property: value;
}
```

E.g.:

HTML

```
HTML:
<div>
<p>Paragraph One</p>
<section>
<p>Paragraph Two</p>
</section>
</div>
CSS:
        div p {
                color: red;
        }
```

Output:

Paragraph One

Paragraph Two

Both paragraphs become red because both are descendants of .

Why is Descendant Selector Important?

- Selects nested elements.
- Useful for styling entire sections.
- Reduces repeated CSS code.

Note: Here also the parent needs to be in the left side of the and the child should be at right side. We can also use class or id names instead of direct element names.

Difference between Direct Child Selector and Descendant Child Selector

Direct Child Selector	Descendant Child Selector
Uses >	Uses space
Selects only direct children	Selects all nested elements
More specific	More flexible

Pseudo Class Selectors

Pseudo Classes Selectors are able to select elements based on their **state** or **user interaction**. They are represented using a **single colon (:)**. It targets specific event especially when we move, click or select certain elements.

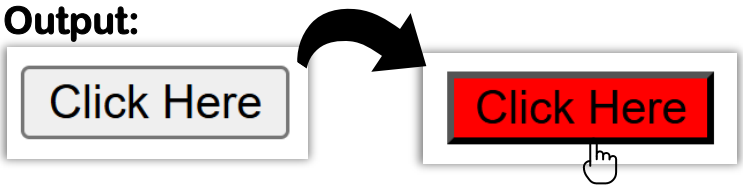
:hover

The **:hover** pseudo class selector applies styles when the mouse pointer moves over an element.

E.g.:

```
button:hover{
        cursor: pointer;
        background-color: red;
        }
```

Output:



Why is it Important?

Provides visual feedback when users move the mouse over an element. It can be applied on any html elements.

:focus

The **:focus** pseudo class applies styles when an input field or other interactive element receives focus.

E.g.:

```
input:focus{
    border: 2px solid blue;
    }
```

Output:

Why is it Important?

Helps users identify the currently active input field. Its only applicable on input tags.

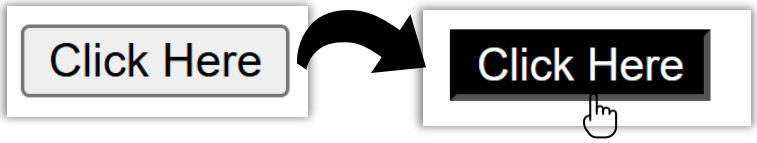
:active

The **:active** pseudo class applies styles while an element is being clicked.

E.g.:

```
button:active{
    color: white;
    background-color: black;
    }
```

Output:



Why is it Important?

Shows that the element is currently being pressed. It can only be applicable on buttons and anchor tag links.

:visited

The **:visited** pseudo class styles hyperlinks that have already been visited by the user.

E.g.:

```
a:visited{
    color: green;
    }
```

Output:

Why is it Important?

Helps users distinguish between visited and unvisited links. Its only applicable on anchor tags.

Pseudo Element Selectors

Pseudo Elements style a specific part of an HTML element rather than the whole element. They are represented using **double colons (::)**. It generally targets specific parts of a html element.

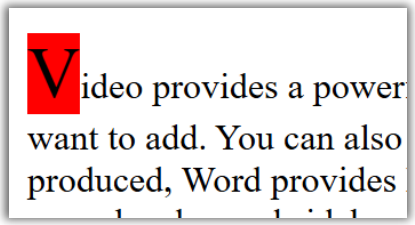
::first-letter

It styles only the first letter of a paragraph.

E.g.:

```
p::first-letter{
    font-size: 32px;
    background-color: red;
    }
```

Output:



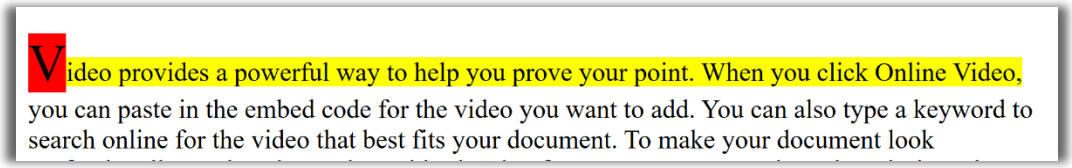
::first-line

It styles only the first line of a paragraph.

E.g.:

```
p::first-line{
    background-color: yellow;
    }
```

Output:



::placeholder

It styles the placeholder text inside input fields.

E.g.:

```
input::placeholder{
   color: brown;
}
```

Output:

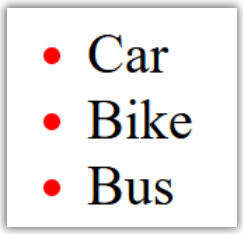
::marker

It styles the bullets or numbers of lists.

E.g.:

```
li::marker{
   color:red;
}
```

Output:



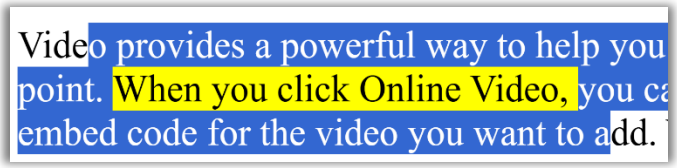
::selection

Styles the text selected by the user.

HTML:

```
<p>Video provides a powerful
way to help you prove your point.
<span> When you click
Online Video, </span>
you can paste in the embed code for the
video you want to add. You can also
type a keyword to search online for
the video that best fits your document.</p>
```

Output:



CSS:

```
span::selection{
   background-color: yellow;
   color: black;
}
```

Difference between Pseudo Class and Pseudo Element

Pseudo Class	Pseudo Element
Uses a single colon (:)	Uses double colons (::)
Styles an element based on its state	Styles a specific part of an element
Example: :hover	Example: ::first-letter

Pseudo Child Selectors

Pseudo Child Selectors are a special type of **Pseudo Class Selectors** that are used to select HTML elements based on their **position among their sibling elements**. Instead of selecting an element using its **id**, **class**, or **tag name**, Pseudo Child Selectors select elements according to their order inside the parent element. They are represented using a **single colon (:)**.

Why are Pseudo Child Selectors Important?

- Select elements based on their position.
- Reduce the need for extra class or id attributes.
- Make CSS cleaner and easier to maintain.
- Help create alternating colors, layouts and designs.
- Improve the appearance of lists and tables.

:first-child Selector

The **:first-child** selector selects the **first child element** of its parent.

Syntax

```
selector:first-child{
        property:value;
        }
```

E.g.:

HTML:

```
<ul>
   <li>HTML</li>
   <li>CSS</li>
   <li>JavaScript</li>
</ul>
```

CSS:

```
li:first-child{
        color: orange;
        }
```

Output:

- HTML
- CSS
- JavaScript

Only **HTML** becomes orange because it is the first child of the <ul> element.

Note: The :first-child selector works only if the selected element is the **first child** of its parent.

:last-child Selector

The **:last-child** selector selects the **last child element** of its parent.

Syntax

```
selector:last-child{
        property:value;
        }
```

E.g.: HTML:

```
<ul>
   <li>HTML</li>
   <li>CSS</li>
   <li>JavaScript</li></ul>
```

Output:

- HTML
- CSS
- JavaScript

CSS:

```
li:last-child{
        color: red;
        }
```

Only **JavaScript** becomes red because it is the last child of the <ul> element.

Note: The :last-child selector works only if the selected element is the **last child** of its parent.

:nth-child() Selector

The **:nth-child()** selector selects an element according to its **position (number)** inside its parent element. The position starts from **1**, not **0**.

Syntax

```
selector:nth-child(number){
        property:value;
}
```

E.g.:

HTML:

```
<ul>
   <li>HTML</li>
   <li>CSS</li>
   <li>JavaScript</li>
   <li>Bootstrap</li>
</ul>
```

CSS:

```
li:nth-child(2){
        color: blue;
        }
```

Output:

- HTML
- CSS
- JavaScript
- Bootstrap

Only **CSS** becomes blue because it is the **second child**.

Note: The numbering in :nth-child() always starts from **1**. It can accept a specific number, **odd**, or **even** as its value.

Difference between :first-child, :last-child and :nth-child()

:first-child	:last-child	:nth-child()
Selects the first child element.	Selects the last child element.	Selects any child element based on its position.
Only one element is selected.	Only one element is selected.	One or multiple elements can be selected.
Example: li:first-child	Example: li:last-child	Example: li:nth-child(3)